

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT
KHOA TOÁN KINH TẾ



HỌC PHẦN: PHÂN TÍCH DỮ LIỆU
ĐỀ TÀI

**ỨNG DỤNG MACHINE LEARNING TRONG DỰ BÁO
DOANH SỐ BÁN HÀNG TẠI HỆ THỐNG BIG MART**



Sinh viên
Phạm Phương Thy
K234131511

Giảng viên hướng dẫn
ThS. Trương Quang Nhật

MỤC LỤC

PHỤ LỤC BẢNG	3
PHỤ LỤC HÌNH.....	3
LỜI CẢM ƠN.....	5
LỜI CAM KẾT	6
PHẦN I. GIỚI THIỆU	7
1.1. Tổng quan về đề tài nghiên cứu	7
1.1.1. Bối cảnh và lý do chọn đề tài	7
1.1.2. Mục tiêu phân tích và câu hỏi định hướng	8
1.1.3. Đối tượng và phạm vi phân tích	8
1.2. Giới thiệu bộ dữ liệu.....	9
1.2.1. Nguồn dữ liệu.....	9
1.2.2. Kích thước dữ liệu	10
1.2.3. Mô tả các biến trong dữ liệu	11
1.2.4. Thống kê mô tả và khoảng giá trị của các biến	12
1.2.5. Kiểm tra tính tương quan giữa các biến định lượng	16
PHẦN II. GIỚI THIỆU BÀI TOÁN	19
2.1. Xác định loại bài toán và mô hình lựa chọn	19
2.2. Sơ đồ quy trình thực hiện.....	20
PHẦN III. XỬ LÝ DỮ LIỆU	23
3.1. Chia dữ liệu Train – Test.....	23
3.2. Lựa chọn dữ liệu phân tích	23
3.3. Kiểm tra và loại bỏ dữ liệu trùng lặp	24
3.4. Kiểm tra và xử lý biến ngoại lai.....	24
3.5. Kiểm tra và xử lý dữ liệu rỗng.....	25
3.5.1. Các bước xử lý dữ liệu thiếu của biến <i>Item_Weight</i>	26
3.5.2. Các bước xử lý dữ liệu thiếu của biến <i>Outlet_Size</i>	30
3.5.3. Xử lý giá trị bằng 0 bất hợp lý trong biến <i>Item_Visibility</i>	33
3.6. Kiểm tra tính hợp lý của dữ liệu	37
3.6.1. Xử lý tính không hợp lý của biến <i>Item_Fat_Content</i>	37
3.6.2. Điều chỉnh hàm lượng chất béo cho sản phẩm không tiêu dùng.....	38
3.7. Xử lý biến thời gian và tạo biến mới	41
3.8. Loại bỏ các biến không cần thiết.....	42
3.9. Chuẩn bị dữ liệu cho mô hình	43
3.9.1. Áp dụng các bước xử lý dữ liệu cho tập train – test	43
3.9.2. Xử lý biến phân loại (biến định tính) để mô hình hiểu	44

PHẦN IV. MÔ HÌNH HOÁ DỮ LIỆU	47
4.1. Cơ sở lý thuyết mô hình	47
4.1.1. <i>Linear Regression</i>	47
4.1.2. <i>Lasso & Ridge Regression</i>	48
4.1.3. <i>Random Forest</i>	49
4.1.4. <i>XGBoost</i>	52
4.2. Triển khai mô hình trên dữ liệu	55
4.3. Tối ưu siêu tham số	61
4.3.1. <i>Cross-validation</i>	61
4.3.2. <i>RandomizedSearchCV</i>	62
4.3.3. <i>GridSearchCV</i>	62
PHẦN V. ĐÁNH GIÁ MÔ HÌNH	64
5.1. Thước đo đánh giá	64
5.1.1. <i>Root Mean Squared Error (RMSE) - Căn bậc hai sai số bình phương trung bình</i>	64
5.1.2. <i>Mean Absolute Error (MAE) - Sai số tuyệt đối trung bình</i>	64
5.1.3. <i>R-squared (R2) - Hệ số xác định</i>	65
5.2. Kết quả dự báo	66
5.3. Kiểm tra độ ổn định của mô hình	68
PHẦN VI. PHÂN TÍCH CÁC YẾU TỐ ẢNH HƯỞNG ĐẾN DOANH SỐ	69
6.1. Phân tích độ quan trọng của biến	69
6.2. So sánh giá trị thực tế và dự đoán	71
PHẦN VII. NGUỒN TÀI LIỆU THAM KHẢO	73

PHỤ LỤC BẢNG

Bảng 1.1. Mô tả các biến trong dữ liệu

PHỤ LỤC HÌNH

Hình 1.1. Sơ đồ cấu trúc phân tầng của dữ liệu

Hình 1.2. Tổng quan về cấu trúc bộ dữ liệu

Hình 1.3. Kiểm tra dữ liệu thiếu của bộ dữ liệu

Hình 1.4. Khoảng giá trị của các biến định lượng

Hình 1.5. Khoảng giá trị của các biến định tính

Hình 1.6. Ma trận tương quan giữa các biến số

Hình 2.1. Sơ đồ quy trình thực hiện

Hình 3.1. Trực quan hoá phân phối và ngoại lai của biến Item_Visibility

Hình 3.2. Kết quả kiểm tra giá trị thiếu trong tập train

Hình 3.3. Kiểm tra trọng lượng của cùng mã sản phẩm DRA59

Hình 3.4. Kiểm tra trọng lượng bao gồm cả khuyết của các mã sản phẩm

Hình 3.5. Kết quả 10 dòng đầu tiên của từ điển (Mã sản phẩm – Trọng lượng)

Hình 3.6. Kết quả từ điển (Nhóm sản phẩm – Trọng lượng)

Hình 3.7. Kết quả xử lý dữ liệu thiếu của biến Item_Weight

Hình 3.8. Mối tương quan giữa Outlet_Type và Outlet_Size

Hình 3.9. Kết quả từ điển (Loại cửa hàng – Quy mô cửa hàng)

Hình 3.10. Kết quả xử lý dữ liệu thiếu của biến Outlet_Size

Hình 3.11. Trực quan hoá phân phối và ngoại lai của biến Item_Visibility

Hình 3.12. Kiểm tra diện tích trưng bày của từng mã sản phẩm

Hình 3.13. Kết quả từ điển (Mã sản phẩm – Trung vị diện tích sản phẩm)

Hình 3.14. Kiểm tra diện tích trưng bày của từng loại sản phẩm

Hình 3.15. Kết quả từ điển (Loại sản phẩm – Trung vị diện tích sản phẩm)

Hình 3.16. Kết quả biến Item_Fat_Content là không có tính hợp lý

Hình 3.17. Kết quả xử lý tính không hợp lý của biến Item_Fat_Content

Hình 3.18. Tần suất Item_Fat_Content theo Item_Type

Hình 3.19. Hàm lượng chất béo của sản phẩm Non-Consumable

Hình 3.20. Kết quả điều chỉnh Item_Fat_Content cho Non-Consumable

Hình 3.21. Trực quan hóa phân phối và ngoại lai của biến

Outlet_Establishment_Year

Hình 3.22. Tần suất phân bố của biến Outlet_Establishment_Year

Hình 4.1. Sơ đồ về Machine Learning (Ali et al., 2023)

Hình 4.2. Ví dụ về mô hình RandomForest

Hình 4.3. Ví dụ về mô hình XGBoost

Hình 5.1. Kết quả dự báo tổng hợp của các mô hình

Hình 5.2. Kết quả kiểm tra độ ổn định của mô hình

Hình 5.3. So sánh RMSE Cross-Validation và RMSE Test của XGBoost

Hình 6.1. Top 10 biến quan trọng đến doanh thu

Hình 6.2. So sánh doanh số thực và dự đoán

LỜI CẢM ƠN

Tôi xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn, người đã tận tình chỉ dẫn, góp ý và hỗ trợ chúng tôi trong suốt quá trình thực hiện tiểu luận này. Những kiến thức, kinh nghiệm và lời khuyên quý báu của thầy đã giúp chúng tôi định hướng phương pháp nghiên cứu, xử lý dữ liệu và trình bày kết quả một cách khoa học và logic.

LỜI CAM KẾT

Tôi xin cam kết rằng toàn bộ nội dung và kết quả trình bày trong tiểu luận này là sản phẩm nghiên cứu của bản thân, được thực hiện một cách trung thực, không sao chép từ bất kỳ nguồn nào mà không trích dẫn rõ ràng. Mọi tài liệu, kết quả số liệu và ý tưởng tham khảo từ các nguồn khác đã được trích dẫn đầy đủ theo chuẩn học thuật. Tôi hoàn toàn chịu trách nhiệm về tính chính xác và trung thực của các thông tin trình bày trong tiểu luận này.

PHẦN I. GIỚI THIỆU

1.1. Tổng quan về đề tài nghiên cứu

1.1.1. Bối cảnh và lý do chọn đề tài

Trong bối cảnh ngoài thị trường bán lẻ hiện tại thì việc dự báo doanh số đóng vai trò quan trọng trong quản lý tồn kho, phân chia nguồn lực và lên chiến lược kinh doanh. Khả năng dự báo nhu cầu tiêu dùng một cách hiệu quả cho phép doanh nghiệp tối ưu hóa tồn kho, giảm thiểu các chi phí và nâng cao mức độ hài lòng của khách hàng thông qua việc sản phẩm luôn sẵn có. Đặc biệt, đối với nhóm hàng thực phẩm, nhất là thực phẩm tươi sống, yêu cầu này càng trở nên cấp thiết trong bối cảnh người Việt Nam ngày càng quan tâm đến vấn đề an toàn thực phẩm.

Theo các phương pháp truyền thống, dự báo chủ yếu dựa trên các mô hình thống kê như hồi quy tuyến tính. Tuy nhiên, các phương pháp này còn nhiều hạn chế thường gặp khi xử lý dữ liệu bán lẻ có tính chất phức tạp bao gồm sự đa dạng về chủng loại sản phẩm, sự khác biệt giữa các cửa hàng và mối quan hệ phi tuyến giữa các yếu tố ảnh hưởng đến doanh số. Những đặc điểm này khiến cho các mô hình tuyến tính khó có thể nắm hết cấu trúc dữ liệu, từ đó làm giảm hiệu quả dự báo.

Sự phát triển của học máy đã mở ra những phương hướng tiếp cận mới trong phân tích dữ liệu và dự báo. Các mô hình học máy có khả năng xử lý các mối quan hệ phi tuyến, xử lý dữ liệu có tính không đồng nhất và cả các biến đầu vào. Nhờ đó, các phương pháp này ngày càng được ứng dụng rộng rãi trong lĩnh vực bán lẻ nhằm nâng cao hiệu quả dự báo.

Tuy nhiên, việc áp dụng các mô hình học máy vào dữ liệu bán lẻ vẫn tồn tại những khó khăn nhất định, đặc biệt khi dữ liệu có cấu trúc phức tạp, thiếu và không bao gồm yếu tố thời gian. Do đó, việc lựa chọn các mô hình phù hợp để khai thác hiệu quả dữ liệu là một vấn đề cần được nghiên cứu.

Xuất phát từ những vấn đề trên, nghiên cứu này được thực hiện nhằm xây dựng và đánh giá các mô hình học máy trong bài toán **ỨNG DỤNG MACHINE LEARNING TRONG DỰ BÁO DOANH SỐ BÁN HÀNG TẠI HỆ THỐNG BIG MART**. Nghiên cứu không chỉ hướng đến việc nâng cao hiệu quả dự báo mà còn tập trung

làm rõ vai trò của các yếu tố ảnh hưởng đến doanh số, từ đó hỗ trợ doanh nghiệp trong việc đưa ra các quyết định kinh doanh phù hợp.

1.1.2. Mục tiêu phân tích và câu hỏi định hướng

Mục tiêu phân tích:

- Xây dựng mô hình dự báo doanh số bán hàng dựa trên các đặc điểm của sản phẩm và cửa hàng, nhằm ước tính doanh số trong các điều kiện khác nhau.
- Nghiên cứu và nhận biết các thuộc tính quan trọng của sản phẩm và cửa hàng, bao gồm các yếu tố như giá niêm yết, loại hình và quy mô cửa hàng, nhằm làm rõ các động lực kinh tế chi phối doanh số bán hàng trong hệ thống bán lẻ.
- Phân tích độ quan trọng của các biến đầu vào thông qua mô hình học máy nhằm đánh giá mức độ tác động của từng đối tượng trong quá trình dự báo, từ đó hỗ trợ phân tích mô hình và nâng cao tính thực tế trong kết quả nghiên cứu.

Câu hỏi định hướng:

- Làm thế nào để xây dựng một mô hình học máy có khả năng dự báo chính xác doanh số bán hàng dựa trên các đặc điểm của sản phẩm và cửa hàng?
- Những yếu tố nào của sản phẩm và cửa hàng có ảnh hưởng đến doanh số bán hàng trong hệ thống Big Mart?
- Mô hình nào trong số các phương pháp được lựa chọn (Linear Regression, Random Forest, XGBoost) đem lại hiệu quả dự báo tốt nhất đối với bộ dữ liệu nghiên cứu?
- Các đặc trưng đầu vào đóng vai trò như thế nào trong quá trình dự báo và yếu tố nào có mức độ ảnh hưởng lớn nhất đến kết quả mô hình?

1.1.3. Đối tượng và phạm vi phân tích

Đối tượng nghiên cứu:

Đề tài tập trung nghiên cứu các mô hình học máy trong bài toán dự báo doanh số bán hàng, cụ thể là các thuật toán hồi quy như Linear Regression, Random Forest và XGBoost. Và bên cạnh đó cũng xem xét các đặc điểm của sản phẩm và cửa hàng trong hệ thống Big Mart, bao gồm các biến liên quan đến giá, đặc điểm sản phẩm, loại hình và quy mô cửa hàng, nhằm phân tích mối quan hệ giữa các yếu tố này với doanh số bán hàng.

Phạm vi nghiên cứu:

Phạm vi của đề tài được giới hạn trong việc sử dụng bộ dữ liệu Big Mart Sales Prediction để xây dựng và đánh giá các mô hình dự báo doanh số. Nghiên cứu tập trung vào dữ liệu dạng cắt ngang (cross-sectional data), không quan trọng yếu tố thời gian.

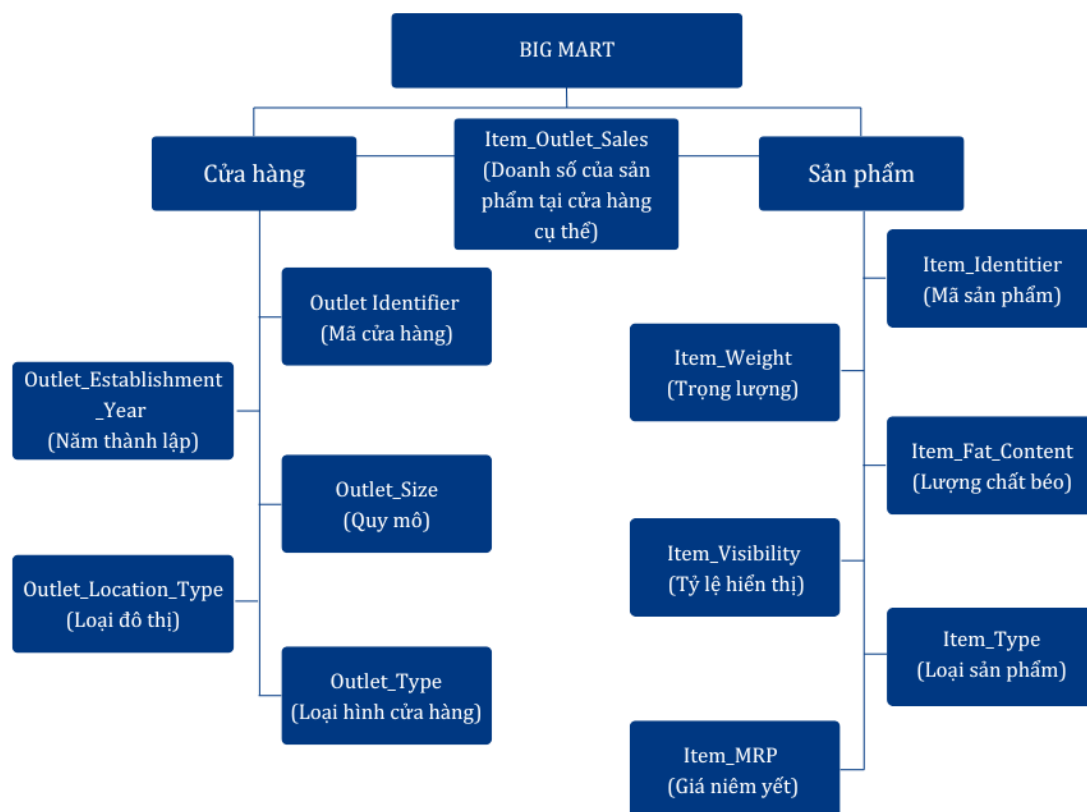
Về mặt phương pháp, đề tài chỉ triển khai một số mô hình học máy phổ biến và phù hợp với bài toán hồi quy, không mở rộng sang các phương pháp nâng cao khác. Quá trình đánh giá mô hình được thực hiện thông qua các thước đo định lượng như R^2 , MSE và RMSE, kết hợp với phân tích độ quan trọng của biến nhằm giải thích kết quả dự báo.

1.2. Giới thiệu bộ dữ liệu

1.2.1. Nguồn dữ liệu

Bộ dữ liệu được sử dụng trong nghiên cứu này là **Big Mart Sales Prediction Datasets** được các nhà khoa học dữ liệu tại BigMart đã thu thập dữ liệu bán hàng tại năm 2013 của 1.559 sản phẩm tại 10 cửa hàng thuộc các thành phố khác nhau. Dữ liệu bao gồm thông tin về doanh số bán hàng của các sản phẩm tại hệ thống cửa hàng bán lẻ Big Mart, được xây dựng nhằm phản ánh mối quan hệ giữa kết quả kinh doanh và các đặc điểm liên quan đến sản phẩm như cửa hàng phân phối, giá bán niêm yết sản phẩm,... Dữ liệu được chia thành hai tập riêng biệt, bao gồm tập huấn luyện (train) chứa đầy đủ biến đầu vào và biến mục tiêu, và tập kiểm tra (test) chỉ bao gồm các biến đầu vào, phục vụ cho việc đánh giá khả năng dự báo của mô hình. Cách phân chia này cho phép xây dựng mô hình dự báo trong môi trường kiểm soát, đồng thời kiểm định tính tổng quát của mô hình trên dữ liệu chưa từng được quan sát trước đó. Ở bài nghiên cứu này, tôi chỉ dùng tập huấn luyện (train) để phân tích và đánh giá mô hình dự báo.

Một trong những đặc điểm quan trọng nhất của bộ dữ liệu là cấu trúc phân tầng như sau:



Hình 1.1. Sơ đồ cấu trúc phân tầng của dữ liệu

Bên cạnh đó, dữ liệu thể hiện tính không đồng nhất rõ rệt khi mà các cửa hàng tổ chức khác nhau về quy mô, vị trí và loại hình hoạt động, trong khi các sản phẩm cũng có đặc điểm và mức giá khác nhau. Nó dẫn đến sự khác biệt lớn trong doanh số và đòi hỏi mô hình phải có khả năng học được các mối quan hệ phi tuyến.

Một điểm cần chú ý là bộ dữ liệu không bao gồm yếu tố thời gian, do đó không phản ánh sự biến động của doanh số theo chu kỳ thời gian. Hệ quả là bài toán được tiếp cận dưới dạng hồi quy chéo, trong đó mô hình không phụ thuộc vào xu hướng thời gian mà sẽ học được mối quan hệ giữa các biến đầu vào và doanh số. Cách này cho phép làm rõ vai trò của từng yếu tố trong việc hình thành doanh số, đồng thời giảm bớt sự phụ thuộc vào các yếu tố biến động theo thời gian.

Từ những đặc điểm trên ta có thể thấy bộ dữ liệu tạo điều kiện thuận lợi cho việc nghiên cứu và so sánh các mô hình học máy trong bài toán dự báo doanh số, đồng thời cung cấp cơ sở để phân tích các yếu tố ảnh hưởng đến hiệu quả kinh doanh trong lĩnh vực bán lẻ.

1.2.2. Kích thước dữ liệu

Cơ sở dữ liệu trong bài nghiên cứu này bao gồm 1559 sản phẩm và 10 cửa hàng ở các thành phố khác nhau trong năm 2013. Mẫu dữ liệu bao gồm 8523 dòng quan sát và 12 biến bao gồm biến phụ thuộc và biến độc lập.

1.2.3. Mô tả các biến trong dữ liệu

Nhằm phục vụ cho quá trình phân tích và chạy dự báo mô hình, việc hiểu rõ bản chất và ý nghĩa của các biến trong bộ dữ liệu là cần thiết trong nghiên cứu. Dưới đây là bảng mô tả các biến trình bày chi tiết bao gồm tên biến, giải thích ý nghĩa và kiểu dữ liệu ban đầu.

STT	Tên biến	Giải thích ý nghĩa	Kiểu dữ liệu ban đầu
1	Item_Identifier	Mã định danh duy nhất của sản phẩm	Object
2	Item_Weight	Trọng lượng của sản phẩm	Float64
3	Item_Fat_Content	Sản phẩm có ít béo hay không	Object
4	Item_Visibility	% tổng diện tích trưng bày của tất cả sản phẩm trong một cửa hàng được phân bổ cho sản phẩm cụ thể đó	Float64
5	Item_Type	Danh mục mà sản phẩm thuộc về	Object
6	Item_MRP	Giá bán lẻ tối đa (giá niêm yết) của sản phẩm	Float64
7	Outlet_Identifier	Mã định danh duy nhất của cửa hàng	Object
8	Outlet_Establishment_Year	Năm cửa hàng được thành lập	Int64

9	Outlet_Size	Quy mô của cửa hàng tính theo diện tích mặt bằng bao phủ	Object
10	Outlet_Location_Type	Loại hình thành phố nơi cửa hàng tọa lạc	Object
11	Outlet_Type	Cửa hàng đó chỉ là một cửa hàng tạp hóa hay là một loại hình siêu thị nào đó	Object
12	Item_Outlet_Sales	Doanh số của sản phẩm tại cửa hàng cụ thể đó. Đây là biến phụ thuộc cần được dự báo	Float64

Bảng 1.1. Mô tả các biến trong dữ liệu

Phân loại và vai trò của các biến số:

- Biến phụ thuộc: Item_Outlet_Sales – Thước đo chính phản ánh kết quả dự báo
- Biến độc lập định lượng: Bao gồm Item_Weight, Item_Visibility, Item_MRP và Outlet_Establishment_Year (sẽ được chuyển đổi thành Outlet_Age); dùng trực tiếp trong mô hình hồi quy.
- Biến độc lập định tính: Bao gồm Item_Fat_Content, Item_Type, Outlet_Size, Outlet_Location_Type và Outlet_Type; sẽ được xử lý thông qua mã hóa (encoding) để đưa vào mô hình học máy và phục vụ phân tích so sánh giữa các nhóm.
- Biến định danh: Bao gồm Item_Identifier và Outlet_Identifier; các biến này chỉ mang tính chất nhận diện và không có ý nghĩa phân tích, do đó được loại bỏ trong quá trình xây dựng mô hình.

1.2.4. Thống kê mô tả và khoảng giá trị của các biến

Bộ dữ liệu Big Mart Sales Prediction Datasets bao gồm 8523 quan sát, trải dài từ chỉ số 0 đến 8522. Về mặt cấu trúc, dữ liệu được tổ chức thành 12 cột thuộc nhiều kiểu dữ liệu khác nhau, phản ánh tính đa dạng của thông tin trong quản lý bán lẻ. Sự phân bố các kiểu dữ liệu cụ thể như sau:

- 7 biến định tính: Lưu trữ các thông tin định danh và phân loại như Item_Identifier, Item_Fat_Content, Item_Type, Outlet_Identifier, Outlet_Size, Outlet_Location_Type, Outlet_Type.

- 5 biến định lượng: Bao gồm 4 cột số thực (float64) và 1 cột số nguyên (int64), phục vụ cho các phép tính toán thống kê và dự báo doanh số: Item_Weight, Item_Visibility, Item_MRP, Outlet_Establishment_Year, Item_Outlet_Sales.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Item_Identifier                       8523 non-null   object
1   Item_Weight                           7060 non-null   float64
2   Item_Fat_Content                       8523 non-null   object
3   Item_Visibility                       8523 non-null   float64
4   Item_Type                             8523 non-null   object
5   Item_MRP                             8523 non-null   float64
6   Outlet_Identifier                     8523 non-null   object
7   Outlet_Establishment_Year            8523 non-null   int64
8   Outlet_Size                           6113 non-null   object
9   Outlet_Location_Type                 8523 non-null   object
10  Outlet_Type                           8523 non-null   object
11  Item_Outlet_Sales                     8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

Hình 1.2. Tổng quan về cấu trúc bộ dữ liệu

Qua quan sát cột Non-Null Count, chúng ta nhận thấy tập dữ liệu chưa thực sự hoàn hảo khi tồn tại sự thiếu hụt thông tin tại hai vị trí quan trọng: Item_Weight và Outlet_Size.

Vì vậy tôi đã chạy lệnh `df.isnull().sum()` để có thể thấy rõ biến Item_Weight thiếu 1463 quan sát và biến Outlet_Size thiếu 2410 quan sát.

	0
Item_Identifier	0
Item_Weight	1463
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	2410
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0
dtype: int64	

Hình 1.3. Kiểm tra dữ liệu thiếu của bộ dữ liệu

Do bộ dữ liệu bao gồm cả biến định lượng và biến định tính, nên khi phân tích thống kê mô tả và khoảng giá trị của các biến, tôi chia thành hai nhóm để dễ quan sát và đánh giá hơn.

Đối với biến định lượng

	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Item_Outlet_Sales
count	7060.000000	8523.000000	8523.000000	8523.000000	8523.000000
mean	12.857645	0.066132	140.992782	1997.831867	2181.288914
std	4.643456	0.051598	62.275067	8.371760	1706.499616
min	4.555000	0.000000	31.290000	1985.000000	33.290000
25%	8.773750	0.026989	93.826500	1987.000000	834.247400
50%	12.600000	0.053931	143.012800	1999.000000	1794.331000
75%	16.850000	0.094585	185.643700	2004.000000	3101.296400
max	21.350000	0.328391	266.888400	2009.000000	13086.964800

Hình 1.4. Khoảng giá trị của các biến định lượng

Biến Item_Weight có giá trị dao động từ 4.55 đến 21.35. Nhìn chung, dữ liệu của biến này phân bố khá đều. Giá trị trung bình khoảng 12.86 và trung vị khoảng 12.60 gần như bằng nhau, cho thấy phân phối của biến khá cân đối, không bị lệch nhiều về một phía.

Biến Item_Visibility có giá trị nằm trong khoảng từ 0.00 đến gần 0.33. Biến này có xu hướng lệch phải thể hiện qua việc giá trị trung bình 0.066 lớn hơn trung

vị 0.053, và tồn tại một số giá trị rất cao tối đa 0.328 nằm xa so với phần lớn dữ liệu. Điều này cũng cho thấy có sự xuất hiện của các giá trị ngoại lai.

Biến Item_MRP có giá trị dao động từ 31.29 đến 266.89, tức là khoảng giá khá rộng (gần gấp 9 lần giữa giá thấp nhất và cao nhất). Tuy nhiên, khi nhìn vào các mốc tứ phân vị (25%, 50%, 75%), có thể thấy dữ liệu được phân bố khá ổn định và liên tục. Điều này phản ánh đúng thực tế trong siêu thị, khi các sản phẩm có nhiều mức giá khác nhau, từ rẻ đến cao cấp, nhưng vẫn phân bố hợp lý theo từng phân khúc.

Biến Outlet_Establishment_Year có giá trị dao động từ 1985 đến 2009, Giá trị trung bình của biến khoảng 1997.83 và trung vị là 1999. Hai giá trị này khá gần nhau, cho thấy dữ liệu phân bố tương đối cân đối, không bị lệch quá nhiều về một phía. Điều này có nghĩa là số lượng cửa hàng cũ và mới không quá chênh lệch.

Giá trị của Item_Outlet_Sales dao động rất lớn, từ 33.29 đến hơn 13086.96. Điều này cho thấy độ biến thiên của biến là rất cao. Biến này cũng bị lệch phải rõ rệt vì giá trị trung bình khoảng 2181 cao hơn khá nhiều so với trung vị khoảng 1794. Ngoài ra, khoảng cách lớn giữa mức 75% (khoảng 3101) và giá trị lớn nhất (hơn 13086) cho thấy tồn tại các giá trị ngoại lai ở phía trên. Chính những giá trị ngoại lai này là nguyên nhân khiến mô hình có thể dự đoán kém chính xác ở các mức doanh số cao.

Đối với biến định tính

Đối với các biến định tính, thay vì xem khoảng giá trị như biến số, tôi sử dụng số lượng giá trị khác nhau (*unique*) và tần suất xuất hiện (*freq*) để đánh giá đặc điểm dữ liệu.

	Item_Identifier	Item_Fat_Content	Item_Type	Outlet_Identifier	Outlet_Size	Outlet_Location_Type	Outlet_Type
count	8523	8523	8523	8523	6113	8523	8523
unique	1559	5	16	10	3	3	4
top	FDW13	Low Fat	Fruits and Vegetables	OUT027	Medium	Tier 3	Supermarket Type1
freq	10	5089	1232	935	2793	3350	5577

Hình 1.5. Khoảng giá trị của các biến định tính

Trước hết, biến Item_Identifier có tới 1559 giá trị khác nhau trên tổng 8523 dòng dữ liệu. Điều này cho thấy mỗi sản phẩm gần như có một mã riêng biệt. Tuy

nhiên, đây chỉ là mã định danh nên không mang nhiều ý nghĩa về mặt nội dung, mà chủ yếu dùng để phân biệt các sản phẩm với nhau.

Biến Item_Fat_Content có 5 giá trị khác nhau, trong đó giá trị phổ biến nhất là Low Fat với 5089 lần xuất hiện. Điều này cho thấy phần lớn sản phẩm thuộc nhóm ít béo.

Biến Item_Type có 16 nhóm khác nhau, phản ánh sự đa dạng về loại sản phẩm trong siêu thị. Trong đó, nhóm Fruits and Vegetables xuất hiện nhiều nhất với 1232 lần, cho thấy đây là nhóm hàng chiếm tỷ trọng lớn trong dữ liệu.

Biến Outlet_Identifier có 10 giá trị khác nhau, tương ứng với 10 cửa hàng. Giá trị xuất hiện nhiều nhất là OUT027 với 935 lần. Điều này cho thấy dữ liệu không phân bố đều giữa các cửa hàng, mà có sự chênh lệch về số lượng quan sát.

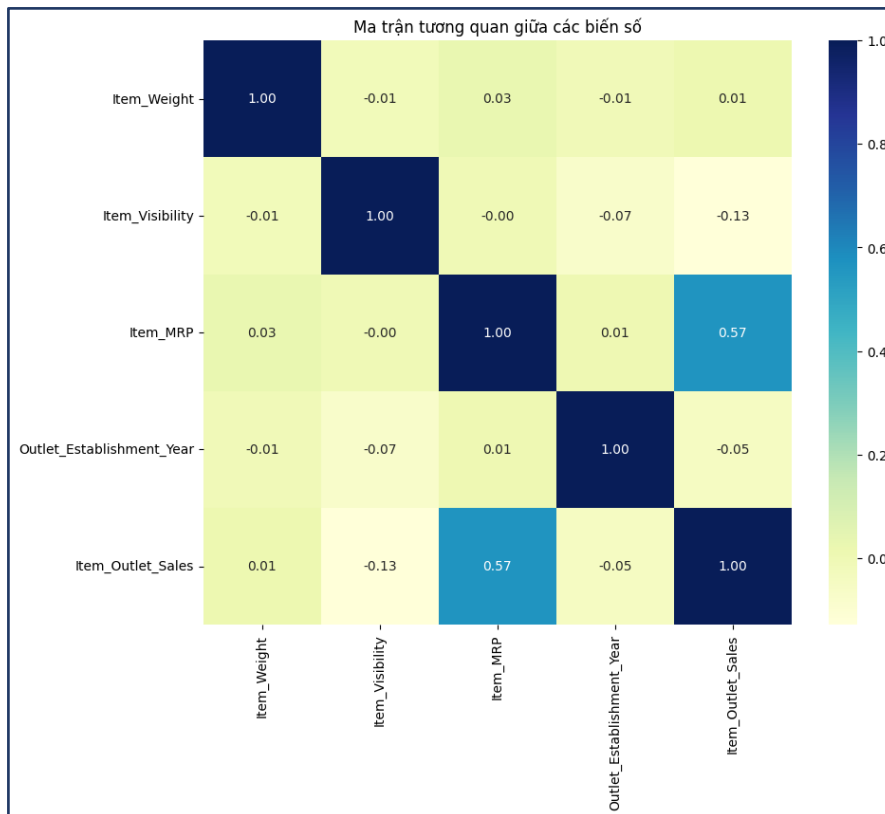
Biến Outlet_Size có 3 nhóm giá trị, trong đó Medium là phổ biến nhất với 2793 lần xuất hiện. Tuy nhiên, số lượng quan sát của biến này chỉ là 6113, thấp hơn tổng số dòng dữ liệu, cho thấy biến đang bị thiếu dữ liệu khá nhiều và cần được xử lý.

Biến Outlet_Location_Type có 3 nhóm, trong đó Tier 3 chiếm tỷ trọng lớn nhất với 3350 lần. Điều này cho thấy phần lớn cửa hàng nằm ở khu vực cấp độ 3.

Cuối cùng, biến Outlet_Type có 4 loại cửa hàng khác nhau, trong đó Supermarket Type 1 chiếm đa số với 5577 lần xuất hiện. Điều này cho thấy loại cửa hàng này đóng vai trò chủ đạo trong hệ thống.

1.2.5. Kiểm tra tính tương quan giữa các biến định lượng

Trong phần này, chúng tôi tiến hành kiểm tra tính tương quan giữa các biến định lượng trong bộ dữ liệu nhằm đánh giá mối quan hệ tuyến tính giữa các yếu tố và doanh số bán lẻ. Dựa trên ma trận tương quan (Heatmap) được xây dựng, hệ số tương quan Pearson được sử dụng, có giá trị từ -1 (tương quan nghịch hoàn toàn) đến 1 (tương quan thuận hoàn toàn).



Hình 1.6. Ma trận tương quan giữa các biến số

Kết quả cho thấy điểm đáng chú ý nhất là mối quan hệ giữa Item_MRP và Item_Outlet_Sales, với hệ số tương quan đạt 0.57, thể hiện mức tương quan thuận trung bình đến mạnh. Điều này cho thấy khi giá bán lẻ tối đa của sản phẩm tăng lên, doanh số bán hàng cũng có xu hướng tăng. Hiện tượng này khá logic trong kinh doanh: Các sản phẩm có giá trị cao hơn thường đóng góp doanh số lớn hơn trên mỗi đơn vị sản phẩm, hoặc khách hàng sẵn sàng chi trả nhiều hơn cho những sản phẩm có mức giá niêm yết cao.

Trong khi đó, hầu hết các cặp biến định lượng khác gần như không có tương quan, với hệ số dao động từ -0.13 đến 0.03, cho thấy chúng gần như độc lập tuyến tính.

Cụ thể:

- Item_Weight (trọng lượng sản phẩm) gần như không liên quan đến doanh số (0.01) hoặc giá bán (0.03), nghĩa là trọng lượng sản phẩm không quyết định sản phẩm bán chạy hay đắt tiền.

- Outlet_Establishment_Year (năm thành lập cửa hàng) có tương quan rất yếu với doanh số (-0.05), ám chỉ rằng tuổi đời cửa hàng không phải là yếu tố quyết định trực tiếp.

- Item_Visibility (độ hiển thị) có tương quan nghịch nhẹ với doanh số (-0.13), một kết quả hơi bất ngờ so với lý thuyết. Điều này có thể phản ánh thực tế rằng các sản phẩm phổ biến chiếm diện tích trưng bày lớn nhưng giá trị thấp, hoặc chiến lược sắp xếp sản phẩm chưa tối ưu.

Tóm lại, phân tích ma trận tương quan cho thấy doanh số phụ thuộc mạnh vào giá bán (MRP), trong khi các yếu tố vật lý của sản phẩm (trọng lượng, độ hiển thị) và thời gian thành lập cửa hàng không có tác động trực tiếp đáng kể. Đây là cơ sở để xác định các biến quan trọng trong mô hình dự báo doanh số bán lẻ.

PHẦN II. GIỚI THIỆU BÀI TOÁN

2.1. Xác định loại bài toán và mô hình lựa chọn

Dựa trên các tài liệu và đặc điểm của bộ dữ liệu Big Mart Sales, bài toán này thuộc nhóm học máy có giám sát và cụ thể là bài toán hồi quy.

Vì đây là học máy có giám sát vì dữ liệu đã có sẵn cả đầu vào và đầu ra. Cụ thể, chúng ta có các thông tin về sản phẩm và cửa hàng (biến đầu vào), đồng thời cũng có sẵn doanh số bán hàng trong quá khứ (Item_Outlet_Sales) để làm đáp án cho mô hình học.

Đây là bài toán hồi quy vì mục tiêu cần dự đoán là một giá trị số liên tục (doanh số), chứ không phải là các nhóm hay nhãn rời rạc như trong bài toán phân loại. Nói đơn giản, mô hình sẽ cố gắng dự đoán một con số doanh thu cụ thể dựa trên nhiều yếu tố khác nhau.

Mục tiêu chính của bài toán là xây dựng một mô hình có thể dự đoán doanh số của từng sản phẩm tại từng cửa hàng, dựa trên các đặc điểm như giá bán, loại sản phẩm, trọng lượng, quy mô cửa hàng hay vị trí cửa hàng.

Đây là bài toán dự báo doanh số trong đó mô hình học cách liên kết giữa đặc điểm sản phẩm – cửa hàng và doanh thu để đưa ra dự đoán chính xác nhất. Các chỉ số như RMSE, MAE, R^2 được sử dụng để đánh giá mức độ dự đoán của mô hình có gần với thực tế hay không.

Về lựa chọn mô hình, trước hết tôi sử dụng Linear Regression làm mô hình cơ sở. Mô hình này đơn giản, dễ hiểu và giúp thiết lập một mức chuẩn để so sánh với các mô hình phức tạp hơn.

Tôi sử dụng Lasso và Ridge Regression nhằm áp dụng kỹ thuật chính quy hóa. Các phương pháp này giúp giảm hiện tượng quá khớp khi dữ liệu có nhiều biến sau khi mã hóa, đồng thời Lasso còn có khả năng tự động loại bỏ các biến không quan trọng giúp mô hình gọn hơn. Khi các biến đầu vào tương quan mạnh, các hệ số của hồi quy tuyến tính thông thường sẽ trở nên rất không ổn định. Hồi quy Ridge được thiết kế đặc biệt để ổn định các hệ số này, giúp dự báo doanh số chính xác và bền bỉ hơn khi đối mặt với các biến có sự tương quan.

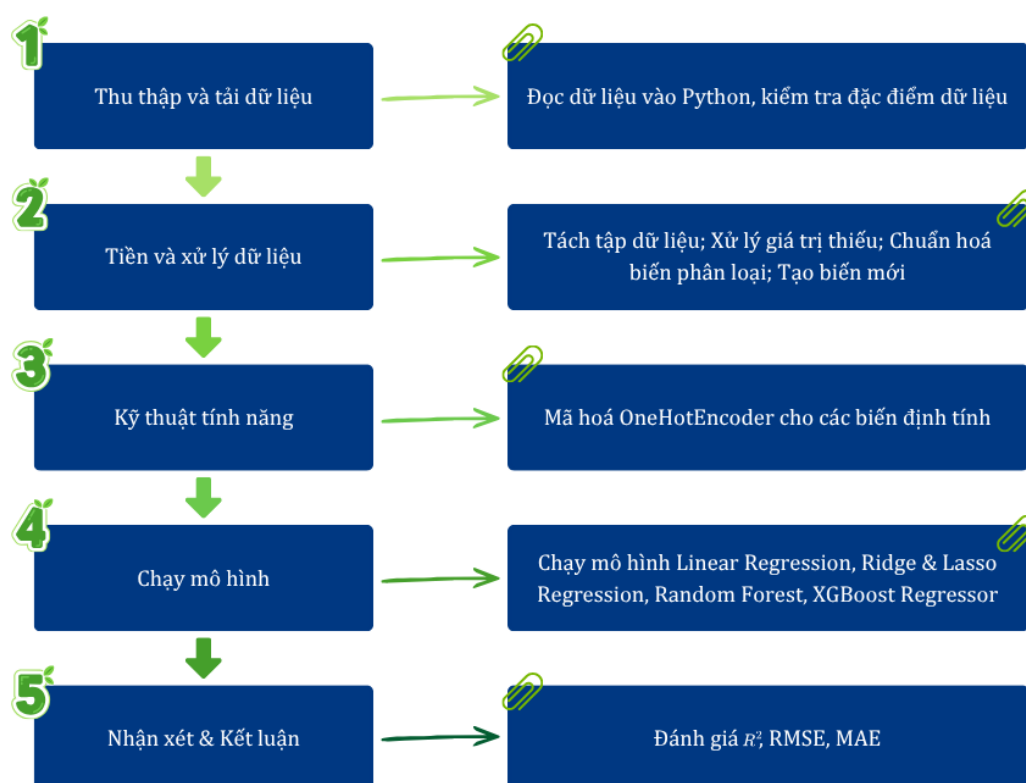
Random Forest được tôi chọn vì là một phương pháp học máy kết hợp (Ensemble) dựa trên kỹ thuật Bagging. Mô hình này xây dựng nhiều cây quyết định độc lập và lấy trung bình kết quả, từ đó giúp giảm sai số và tăng tính ổn định.

Cuối cùng, XGBoost là mô hình nâng cao thuộc nhóm Boosting. Thuật toán này xây dựng các cây quyết định theo dạng tuần tự, trong đó mỗi cây mới sẽ tập trung sửa lỗi của các cây trước. Đồng thời, XGBoost tích hợp các kỹ thuật chính quy hóa mạnh mẽ, giúp đạt độ chính xác cao và thường cho kết quả tốt nhất trong các bài toán dự báo thực tế.

Tóm lại, việc kết hợp nhiều mô hình từ đơn giản đến nâng cao giúp đánh giá toàn diện hiệu quả dự báo, đồng thời lựa chọn được mô hình tối ưu nhất cho bài toán.

2.2. Sơ đồ quy trình thực hiện

Để đảm bảo tính khách quan và độ chính xác của kết quả, nghiên cứu được thực hiện dựa trên quy trình học máy chuẩn hóa. Các bước triển khai chi tiết được trình bày dưới đây:



Hình 2.1. Sơ đồ quy trình thực hiện

Thu thập và tải dữ liệu

Sử dụng các thư viện phân tích dữ liệu cơ bản như numpy, pandas và các công cụ trực quan hóa như matplotlib, seaborn. Các thư viện học máy từ sklearn và xgboost cũng được tích hợp để phục vụ mô hình hóa.

Dữ liệu Big Mart Sales Prediction Datasets được lấy từ Kaggle. Dữ liệu được đọc từ tệp train.csv bằng pandas.

Dữ liệu được tiến hành các bước kiểm tra ban đầu nhằm đánh giá cấu trúc tổng thể, kích thước tập dữ liệu, kiểu dữ liệu của từng biến, đồng thời xác định sự tồn tại của các giá trị thiếu hoặc không hợp lệ.

Tiền và xử lý dữ liệu

Đây là giai đoạn tiền xử lý dữ liệu nhằm đảm bảo chất lượng đầu vào cho mô hình. Trước hết, tập dữ liệu được phân chia thành tập huấn luyện và tập kiểm tra theo một tỷ lệ hợp lý nhằm đảm bảo tính khách quan trong quá trình đánh giá mô hình.

Các giá trị thiếu và bất hợp lý được xử lý bằng những phương pháp phù hợp với đặc điểm của từng biến, bao gồm việc suy luận dựa trên thông tin liên quan, sử dụng các thống kê đại diện như trung bình, trung vị hoặc giá trị xuất hiện nhiều nhất trong từng nhóm dữ liệu.

Các giá trị không hợp lệ hoặc bất thường cũng được điều chỉnh để phản ánh đúng bản chất của dữ liệu. Cuối cùng, dữ liệu được chuẩn hóa nhằm đảm bảo tính nhất quán về định dạng và ý nghĩa giữa các biến, từ đó giúp mô hình học được các quy luật chính xác và ổn định hơn.

Kỹ thuật tính năng

Nghiên cứu còn thực hiện bước xây dựng đặc trưng mới nhằm nâng cao khả năng dự báo của mô hình. Cụ thể là các biến mới được tạo ra dựa trên việc khai thác thông tin tiềm ẩn từ dữ liệu gốc, bao gồm việc khái quát hóa các nhóm sản phẩm, điều chỉnh các thuộc tính để đảm bảo tính logic giữa các biến, cũng như xây dựng các biến phản ánh đặc điểm vận hành như thời gian hoạt động của cửa hàng. Những đặc trưng này góp phần giúp mô hình nắm bắt tốt hơn các yếu tố ảnh hưởng đến biến mục tiêu.

Ngoài ra, các biến phân loại được mã hóa sang dạng số để phù hợp với yêu cầu của các thuật toán học máy. Những cải tiến này giúp mô hình hiểu rõ hơn mối quan hệ giữa các biến đầu vào và biến mục tiêu.

Chạy mô hình

Nhiều thuật toán khác nhau được triển khai nhằm tìm ra mô hình dự báo tối ưu. Trước hết, mô hình Linear Regression được sử dụng làm mô hình cơ sở để thiết lập mức tham chiếu ban đầu. Tiếp theo, các mô hình có điều chuẩn như Ridge Regression và Lasso Regression được áp dụng kết hợp với kỹ thuật tìm kiếm tham số (GridSearchCV) nhằm hạn chế hiện tượng quá khớp.

Các mô hình ensemble mạnh như XGBoost và Random Forest cũng được triển khai và tối ưu hóa thông qua các phương pháp tìm kiếm tham số khác nhau. Việc so sánh nhiều mô hình giúp lựa chọn được phương án có hiệu suất tốt nhất.

Nhận xét & Kết luận

Bước cuối cùng là phân tích và diễn giải các kết quả thu được từ mô hình dự báo. Các thước đo đánh giá như MAE, RMSE và hệ số xác định R^2 được sử dụng để xem xét mức độ phù hợp và độ chính xác của từng mô hình. Đồng thời, kết quả từ các mô hình nâng cao như XGBoost và Random Forest được khai thác để phân tích mức độ quan trọng của từng biến đầu vào, từ đó xác định những yếu tố có ảnh hưởng mạnh nhất đến doanh số bán hàng.

Dựa trên các kết quả nghiên cứu đưa ra các nhận định mang tính thực tiễn, giúp doanh nghiệp hiểu rõ hơn về hành vi tiêu dùng cũng như các yếu tố quyết định doanh thu, qua đó hỗ trợ việc xây dựng chiến lược kinh doanh và tối ưu hóa hiệu quả hoạt động bán lẻ.

PHẦN III. XỬ LÝ DỮ LIỆU

3.1. Chia dữ liệu Train – Test

Ở bài nghiên cứu này, việc chia dữ liệu thành hai tập: tập huấn luyện (train) và tập kiểm tra (test) sẽ được thực hiện trước bước tiền xử lý dữ liệu.

Theo Nicolas Vandeput (2021), tập kiểm tra phải được giữ riêng biệt hoàn toàn khỏi mô hình trong suốt giai đoạn huấn luyện để đảm bảo tính chất của dữ liệu chưa từng thấy (unseen data).

Việc tách tập dữ liệu sớm như vậy nhằm ngăn chặn hiện tượng rò rỉ dữ liệu (data leakage), vốn được định nghĩa là tình trạng mô hình được cung cấp các mảnh thông tin về nhu cầu trong tương lai. Nếu thực hiện các bước tiền xử lý (như tính giá trị trung bình để điền khuyết hoặc mã hóa nhãn) trên toàn bộ dữ liệu trước khi chia, tập huấn luyện sẽ gián tiếp biết trước các đặc điểm phân phối của tập kiểm tra.

Tác giả nhấn mạnh rằng nếu dữ liệu bị rò rỉ, kết quả dự báo đạt được chỉ là một kết quả ảo (mirage) về độ chính xác cao nhưng thực chất là do quá khớp (overfitting) vào các đặc điểm ngẫu nhiên.

Vì vậy, quy trình tách trước - xử lý sau giúp mô phỏng đúng thực tế vận hành: nhà quản lý không bao giờ biết trước các thông số thống kê của dữ liệu ngày mai để điền khuyết cho dữ liệu hôm nay. Cách tiếp cận này đảm bảo rằng mọi quy luật và tham số tiền xử lý chỉ được học từ tập huấn luyện, từ đó giúp đánh giá một cách trung thực và khách quan nhất hiệu suất của mô hình.

Lệnh được sử dụng để tách tập train – test:

```
X = df.drop('Item_Outlet_Sales', axis=1)
y = df['Item_Outlet_Sales']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

Tập huấn luyện (train) sau khi tách có kích thước 5966 quan sát và 11 biến độc lập. Kích thước của tập kiểm tra (test) sau khi tách có 2557 quan sát và 11 biến độc lập.

3.2. Lựa chọn dữ liệu phân tích

Trong bài dự báo doanh số từ dữ liệu Big Mart này, biến phụ thuộc của tôi chính là `Item_Outlet_Sales`, các biến độc lập chính là `Item_Weight`, `Item_Fat_Content`, `Item_Visibility`, `Item_Type`, `Item_MRP`, `Outlet_Establishment_Year`, `Outlet_Size`, `Outlet_Location_Type`, `Outlet_Type`, `Item_Outlet_Sales`.

Ở đây có thể thấy trong bài này tôi sẽ loại bỏ hai biến thuộc tính định danh: `Item_Identifier` và `Outlet_Identifier`. Vì mục tiêu của bộ dữ liệu Big Mart là hiểu các thuộc tính của sản phẩm và cửa hàng đóng vai trò then chốt trong việc tăng doanh số, tuy nhiên hai biến trên chỉ mang tính chất đại diện chứ không chứa thông tin có ý nghĩa giải thích đối với doanh số, nếu giữ lại khi chạy mô hình đôi khi sẽ gây nhiễu. Thay vì nhớ mã sản phẩm cụ thể nào đó, mô hình sẽ nhìn vào loại hàng, trọng lượng, chỉ số MRP, nồng độ chất béo. Điều đó cho thấy được những thuộc tính này mới thật sự quyết định khách hàng có mua sản phẩm đó hay không? Còn về lý do tôi chọn loại mã cửa hàng, bởi vì khi mô hình nhìn vào các yếu tố như: Siêu thị loại nào? Quy mô ra sao? Thành lập bao lâu? Đây sẽ là những yếu tố đại diện cho lưu lượng khách và thói quen mua sắm tại đại điểm đó.

Do vậy, hai biến định danh này sẽ được loại bỏ trong quá trình xây dựng mô hình. Nhưng trong giai đoạn tiền xử lý dữ liệu ban đầu, các biến này vẫn được giữ lại nhằm phục vụ cho việc kiểm tra và xử lý dữ liệu trước khi tiến hành chuẩn bị dữ liệu đầu vào cho mô hình học máy.

3.3. Kiểm tra và loại bỏ dữ liệu trùng lặp

Để kiểm tra dữ liệu trong tập train có bị trùng lặp hay không tôi sử dụng lệnh:

```
X_train_c.duplicated().sum()
```

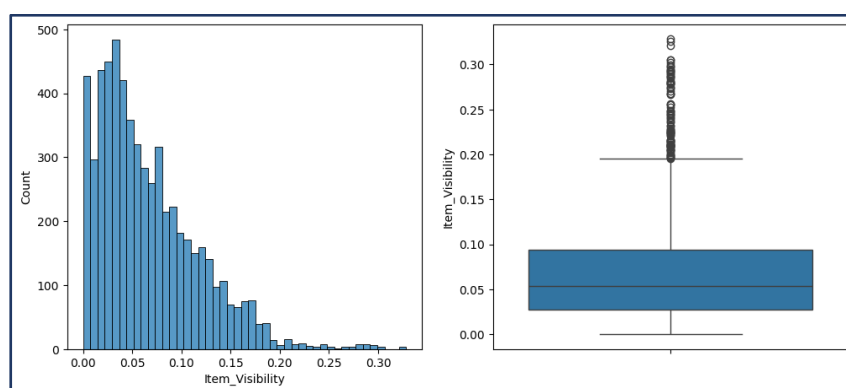
Kết quả tôi nhận được là không có dữ liệu bị trùng lặp `np.int64(0)`. Vì vậy, tôi không cần thực hiện bước xử lý loại bỏ dữ liệu trùng và có thể tiếp tục các bước phân tích tiếp theo.

3.4. Kiểm tra và xử lý biến ngoại lai

Biến ngoại lai là những giá trị cao hoặc thấp một cách bất thường so với phần còn lại của tập dữ liệu. Trong dự báo doanh số, các giá trị này có thể xuất phát từ

sai sót kỹ thuật khi nhập liệu hoặc do các đợt biến động nhu cầu thực tế như khuyến mãi, quảng cáo.

Tôi sử dụng kết hợp biểu đồ Histogram (phân phối tần suất) và Boxplot (biểu đồ hộp) là phương pháp hiệu quả nhất để phát hiện ngoại lai. Sau khi chạy hàng loạt cho các biến định lượng, tôi có kết quả biến Item_Visibility (tỷ lệ diện tích trưng bày sản phẩm) cho thấy sự xuất hiện của nhiều giá trị cao bất thường, nằm xa khỏi phân phối tập trung của dữ liệu.



Hình 3.1. Trực quan hoá phân phối và ngoại lai của biến Item_Visibility

Tuy nhiên, trong bài này tôi không loại bỏ các giá trị ngoại lai này, vì những lý do sau:

- Trong bán lẻ, những sản phẩm có độ hiển thị cao thường là sản phẩm được ưu tiên trưng bày để tăng doanh số. Vì vậy, các giá trị lớn này không phải là lỗi mà phản ánh đúng thực tế.
- Biến Item_Visibility là một trong những yếu tố quan trọng ảnh hưởng đến doanh thu. Nếu loại bỏ các giá trị này, mô hình sẽ mất đi những trường hợp “đặc biệt” nhưng rất quan trọng.

3.5. Kiểm tra và xử lý dữ liệu rỗng

Như đã phân tích ở phần trước, sau khi tiến hành tách dữ liệu thành tập huấn luyện (train) và tập kiểm tra (test), bước tiếp theo là kiểm tra tình trạng dữ liệu thiếu nhằm chuẩn bị tốt cho quá trình xây dựng mô hình.

Kết quả kiểm tra dữ liệu thiếu trên tập huấn luyện được trình bày như sau:

	0
Item_Identifier	0
Item_Weight	1023
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	1690
Outlet_Location_Type	0
Outlet_Type	0
dtype: int64	

Hình 3.2. Kết quả kiểm tra giá trị thiếu trong tập train

Kết quả cho thấy biến Item_Weight bị thiếu 1023 quan sát, trong khi biến Outlet_Size bị thiếu 1690 quan sát. Điều này cho thấy dữ liệu vẫn tồn tại các giá trị khuyết và cần được xử lý trước khi đưa vào mô hình học máy.

3.5.1. Các bước xử lý dữ liệu thiếu của biến Item_Weight

Ở đây tôi không dùng một giá trị trung bình cho tất cả dữ liệu, mà lại xử lý theo nhiều bước khác nhau vì tôi muốn dữ liệu trở nên hợp lý và gần với thực tế hơn. Nói theo cách không phức tạp chính là mỗi sản phẩm sẽ có trọng lượng riêng. Ví dụ: Một chai nước không thể nặng giống một bao gạo. Nên nếu sử dụng dung một giá trị trung bình hay trung vị để điền cho các dữ liệu bị thiếu là điều không hợp lý.

Để củng cố cho lập luận của tôi, tôi đã chạy lệnh để kiểm tra các sản phẩm có cùng mã DRA59 thì có cùng trọng lượng hay không?

```
X_train_c[X_train_c['Item_Identifier'] == 'DRA59'][['Item_Identifier',
'Item_Weight']].sort_values(by='Item_Weight')
```

Và đây là kết quả mà tôi nhận được:

	Item_Identifier	Item_Weight
1876	DRA59	8.27
5609	DRA59	8.27
3927	DRA59	8.27
3962	DRA59	8.27
6057	DRA59	NaN
6179	DRA59	NaN

Hình 3.3. Kiểm tra trọng lượng của cùng mã sản phẩm DRA59

Kết quả cho thấy các dòng có cùng mã sản phẩm đều có cùng một giá trị trọng lượng, từ đó củng cố giả định rằng mỗi mã sản phẩm tương ứng với một trọng lượng cố định.

Tiếp theo, tôi tiếp tục kiểm tra trên toàn bộ dữ liệu để xác nhận rằng các sản phẩm có cùng mã thì có cùng trọng lượng hay không, bằng lệnh sau:

```
X_train_c[['Item_Identifier',  
'Item_Weight']].drop_duplicates().sort_values('Item_Identifier')
```

	Item_Identifier	Item_Weight
1245	DRA12	11.60
4900	DRA24	19.35
6057	DRA59	NaN
1876	DRA59	8.27
3733	DRB01	NaN
...	...	...
411	NCZ42	10.50
2084	NCZ53	NaN
6879	NCZ53	9.60
7926	NCZ54	14.65
1896	NCZ54	NaN

2401 rows x 2 columns

Hình 3.4. Kiểm tra trọng lượng bao gồm cả khuyết của các mã sản phẩm

Nhưng nếu trường hợp trong tập huấn luyện (train) không có mã sản phẩm đó mà tập kiểm tra (test) lại xuất hiện mã sản phẩm mới, lúc này tôi sẽ sử dụng giá trị trung vị của trọng lượng theo từng nhóm sản phẩm. Việc tôi sử dụng giá trị trung vị theo nhóm sản phẩm (Item_Type) là vì trọng lượng là một đặc tính vật lý gắn liền với bản chất của hàng hóa. Các sản phẩm trong cùng một nhóm (như thực phẩm, đồ uống hay hàng không tiêu dùng) thường có dải trọng lượng tương đồng nhau hơn là khi so sánh với các nhóm khác.

Từ lập luận trên, có thể thấy rằng việc điền khuyết cần được thực hiện theo từng bước để đảm bảo tính hợp lý của dữ liệu. Cụ thể, quy trình xử lý được thực hiện như sau:

Bước 1: Điền theo mã sản phẩm (Item_Identifier)

Vì cùng một sản phẩm thì sẽ có cùng trọng lượng như đã trình bày và chứng minh ở trên, nên bài nghiên cứu này dùng trọng lượng có sẵn của sản phẩm đó để điền vào những dòng bị thiếu.

Đầu tiên, tôi sẽ tạo một bảng tra cứu nó sẽ gom nhóm dữ liệu theo Item_Identifier và tính toán giá trị trọng lượng tương ứng cho từng mã. Vì mỗi sản phẩm cụ thể chỉ có một trọng lượng vật lý duy nhất, bảng này sẽ trở thành một danh sách "chuẩn" chứa cặp (Mã sản phẩm - Trọng lượng). Với lệnh:

```
ITEM_ID_WEIGHT_PIVOT = X_train_c.pivot_table(values='Item_Weight',
index='Item_Identifier').reset_index()
```

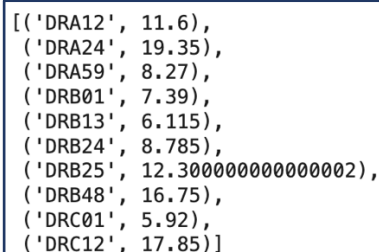
Sau đó tôi sẽ chuyển bảng dữ liệu vừa tạo thành một từ điển trong Python:

```
ITEM_ID_WEIGHT_MAPPING = dict(zip(ITEM_ID_WEIGHT_PIVOT['Item_Identifier'],
ITEM_ID_WEIGHT_PIVOT['Item_Weight']))
```

Việc dùng từ điển giúp máy tính tra cứu trọng lượng của một mã sản phẩm bất kỳ cực nhanh khi thực hiện điền khuyết ở các bước sau

Bây giờ tôi đã có bảng từ điển được tạo ra đúng cấu trúc. Ví dụ, kết quả hiển thị ('DRA12', 11.6) nghĩa là sản phẩm có mã DRA12 nặng 11.6 đơn vị.

```
list(ITEM_ID_WEIGHT_MAPPING.items())[:10]
```



```
[('DRA12', 11.6),
 ('DRA24', 19.35),
 ('DRA59', 8.27),
 ('DRB01', 7.39),
 ('DRB13', 6.115),
 ('DRB24', 8.785),
 ('DRB25', 12.300000000000002),
 ('DRB48', 16.75),
 ('DRC01', 5.92),
 ('DRC12', 17.85)]
```

Hình 3.5. Kết quả 10 dòng đầu tiên của từ điển (Mã sản phẩm – Trọng lượng)

Bước 2: Điền theo nhóm sản phẩm (Item_Type)

Nếu sản phẩm không có dữ liệu trọng lượng nào, tôi sẽ điền bằng giá trị trung vị của nhóm sản phẩm tương ứng.

Nhưng trước khi tôi tạo từ điển theo Item_Type, thì tôi cần đơn giản hoá hơn cho cột Item_Type. Lý do là vì trong dữ liệu ban đầu, cột Item_Type chứa các loại sản phẩm rất chi tiết như Dairy, Meat, Household,... Điều này khiến dữ liệu bị phân tán thành nhiều nhóm nhỏ. Để đơn giản hóa, tôi không dùng trực tiếp cột này nữa

mà tạo lại giá trị cho chính cột `Item_Type` dựa trên mã sản phẩm (`Item_Identifier`).
Cụ thể, tôi lấy 2 chữ cái đầu của mã sản phẩm:

- Nếu bắt đầu bằng **FD** → gán là *Food*
- Nếu bắt đầu bằng **DR** → gán là *Drinks*
- Nếu bắt đầu bằng **NC** → gán là *Non-Consumable*

Để thực hiện được điều này lệnh mà tôi sử dụng:

```
def create_item_type_column(data_frame):  
    data_frame['Item_Type'] = data_frame['Item_Identifier'].str[:2]  
    data_frame['Item_Type'] = data_frame['Item_Type'].map({  
        'FD': 'Food', 'NC': 'Non-Consumable', 'DR': 'Drinks'  
    })  
    return data_frame
```

Như vậy, cột `Item_Type` ban đầu đã được thay thế bằng một cách phân loại đơn giản hơn, chỉ còn 3 nhóm chính.

Tương tự như bước điền theo mã sản phẩm, ở đây tôi cũng tạo một bảng tổng hợp trọng lượng theo loại sản phẩm, sử dụng hàm `index='Item_Type'` để gom nhóm dữ liệu dựa trên biến loại sản phẩm khái quát như (Food, Drinks, Non-Consumable). Sau đó, tôi dùng hàm `aggfunc='median'` để tính trung vị thay vì trung bình. Điều này giúp giá trị đại diện không bị lệch bởi các sản phẩm quá nặng hoặc quá nhẹ trong cùng một nhóm.

Để chuyển bảng dữ liệu vừa tính toán thành dạng từ điển để máy tính có thể tra cứu tức thời tôi sử dụng lệnh:

```
ITEM_TYPE_WEIGHT_MAPPING =  
dict(zip(ITEM_TYPE_WEIGHT_PIVOT['Item_Type'],  
        ITEM_TYPE_WEIGHT_PIVOT['Item_Weight']))
```

Kết quả mà tôi nhận được sẽ có một bảng gồm 3 dòng tương ứng với 3 nhóm sản phẩm và trọng lượng trung vị của chúng.

```
dict_items([('Drinks', 11.8), ('Food', 12.8), ('Non-Consumable', 13.05)])
```

Hình 3.6. Kết quả từ điển (Nhóm sản phẩm – Trọng lượng)

Bước 3: Kết quả

Sau hai bước tạo bảng từ điển (Mã sản phẩm – Trọng lượng) và (Nhóm sản phẩm – Trọng lượng), tôi sẽ xây dựng hàm để điền các dữ liệu Item_Weight thiếu theo theo đúng thứ tự ưu tiên.

Cụ thể, hàm sẽ thực hiện hai bước. Đầu tiên, nó sẽ điền trọng lượng dựa trên mã sản phẩm (Item_Identifier), bằng cách tra cứu từ điển đã tạo. Nếu sản phẩm đã từng xuất hiện trước đó, trọng lượng sẽ được khôi phục chính xác.

Nếu sau bước này vẫn còn giá trị bị thiếu, hàm sẽ tiếp tục điền theo nhóm sản phẩm (Item_Type). Lúc này, mỗi sản phẩm sẽ được gán trọng lượng bằng giá trị trung vị của nhóm như Food, Drinks hoặc Non-Consumable.

Đoạn lệnh thực hiện như sau:

```
def impute_item_weight(data_frame):  
    data_frame.loc[:, 'Item_Weight'] =  
    data_frame.loc[:, 'Item_Weight'].fillna(data_frame.loc[:, 'Item_Identifier'].map(ITEM_ID_WEIGHT_MAPPING))  
    data_frame.loc[:, 'Item_Weight'] =  
    data_frame.loc[:, 'Item_Weight'].fillna(data_frame.loc[:, 'Item_Type'].map(ITEM_TYPE_WEIGHT_MAPPING))  
    return data_frame  
X_train_c = impute_item_weight(X_train_c)
```

Sau khi thực hiện xử lý dữ liệu thiếu, cột Item_Weight đã được điền đầy đủ, không còn giá trị bị thiếu, và dữ liệu đã sẵn sàng để đưa vào mô hình:

	0
Item_Identifier	0
Item_Weight	0
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	1690
Outlet_Location_Type	0
Outlet_Type	0
dtype: int64	

Hình 3.7. Kết quả xử lý dữ liệu thiếu của biến Item_Weight

3.5.2. Các bước xử lý dữ liệu thiếu của biến Outlet_Size

Đối với biến `Outlet_Size`, tôi cũng không sử dụng một giá trị chung để điền cho tất cả dữ liệu bị thiếu, mà thực hiện điền dựa trên loại cửa hàng (`Outlet_Type`) để đảm bảo dữ liệu hợp lý và gần với thực tế hơn.

Có thể hiểu rằng, mỗi loại cửa hàng sẽ có quy mô đặc trưng riêng. Ví dụ, một cửa hàng tạp hóa nhỏ thì khó có thể có quy mô lớn giống như một siêu thị. Nhưng với các siêu thị lớn cũng không thể có quy mô nhỏ như cửa hàng tạp hóa.

Để quan sát xem một loại hình cửa hàng cụ thể (`Outlet_Type`) thường đi kèm với quy mô nào (`Outlet_Size`), tôi sử dụng lệnh:

```
X_train_c.groupby(by=['Outlet_Type', 'Outlet_Size']).size()
```

Kết quả nhận được như sau:

		0
Outlet_Type	Outlet_Size	
Grocery Store	Small	355
	High	649
	Medium	631
Supermarket Type1	Small	1312
	Medium	661
	Medium	668
Supermarket Type2	Medium	661
Supermarket Type3	Medium	668
		dtype: int64

Hình 3.8. Mối tương quan giữa `Outlet_Type` và `Outlet_Size`

Kết quả trả về là một bảng thống kê số lượng (tần suất) xuất hiện của các cặp giá trị này:

- **Grocery Store:** Luôn đi kèm với quy mô **Small**.
- **Supermarket Type 2 & 3:** Luôn đi kèm với quy mô **Medium**.
- **Supermarket Type 1:** Có sự chồng chéo giữa Small, Medium và High, nhưng số lượng Small chiếm ưu thế rõ rệt (1312 lần).

Vì vậy, nếu sử dụng một giá trị chung như giá trị xuất hiện nhiều nhất của toàn bộ dữ liệu để điền cho tất cả các trường hợp bị thiếu, thì sẽ dẫn đến tình trạng các cửa hàng bị gán sai quy mô, làm dữ liệu mất đi tính thực tế. Nên lập luận tôi sẽ sử dụng là điền theo từng loại cửa hàng giúp mỗi cửa hàng được gán một quy mô phù hợp với đặc điểm của nó, từ đó giữ được tính logic và giúp mô hình học chính xác hơn.

Bước 1: Tạo bảng từ điển (Loại cửa hàng – Quy mô cửa hàng)

Đầu tiên, tôi dùng hàm mode để tìm giá trị xuất hiện nhiều nhất trong dữ liệu. Sau đó, tôi tạo bảng thống kê bằng pivot_table, trong đó dữ liệu được gom theo loại cửa hàng (Outlet_Type) và tìm ra quy mô (Outlet_Size) phổ biến nhất cho từng loại.

Cuối cùng, tôi chuyển bảng này thành một từ điển để máy có thể tra cứu nhanh khi điền dữ liệu thiếu.

```
from scipy.stats import mode
OUTLET_TYPE_SIZE_PIVOT = X_train_c.pivot_table(values='Outlet_Size',
index='Outlet_Type',aggfunc=(lambda x : x.mode()[0])).reset_index()
OUTLET_TYPE_SIZE_MAPPING =
dict(zip(OUTLET_TYPE_SIZE_PIVOT['Outlet_Type'],
OUTLET_TYPE_SIZE_PIVOT['Outlet_Size']))
OUTLET_TYPE_SIZE_MAPPING.items()
```

Và bộ quy tắc được tạo ra như sau:

```
dict_items([('Grocery Store', 'Small'), ('Supermarket Type1', 'Small'), ('Supermarket
Type2', 'Medium'), ('Supermarket Type3', 'Medium')])
```

Hình 3.9. Kết quả từ điển (Loại cửa hàng – Quy mô cửa hàng)

Bước 2: Điền các giá trị thiếu từ bảng từ điển

Sau khi đã có bảng từ điển (Loại cửa hàng – Quy mô cửa hàng), tôi xây dựng hàm để điền các giá trị còn thiếu trong cột Outlet_Size.

```
def impute_outlet_size(data_frame):
    data_frame.loc[:, 'Outlet_Size'] =
    data_frame.loc[:, 'Outlet_Size'].fillna(data_frame.loc[:, 'Outlet_Type'
    ].map(OUTLET_TYPE_SIZE_MAPPING))
    return data_frame
X_train_c = impute_outlet_size(X_train_c)
```

Với lệnh này mỗi dòng bị thiếu dữ liệu, máy sẽ nhìn vào loại cửa hàng (Outlet_Type), sau đó tra trong từ điển OUTLET_TYPE_SIZE_MAPPING để lấy ra quy mô phù hợp nhất.

Sau khi thực hiện xử lý dữ liệu thiếu, cột Outlet_Size đã được điền đầy đủ, không còn giá trị bị thiếu, và dữ liệu đã sẵn sàng để đưa vào mô hình:

	0
Item_Identifier	0
Item_Weight	0
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	0
Outlet_Location_Type	0
Outlet_Type	0
dtype: int64	

Hình 3.10. Kết quả xử lý dữ liệu thiếu của biến Outlet_Size

3.5.3. Xử lý giá trị bằng 0 bất hợp lý trong biến Item_Visibility

Mặc dù theo lý thuyết, giá trị 0 có thể được hiểu là sản phẩm không được trưng bày, nhưng trong ngữ cảnh của bộ dữ liệu Big Mart, dựa vào nguồn tài liệu tôi có thể xác định đây là một vấn đề về dữ liệu khuyết vì mâu thuẫn logic với doanh số. Biến Item_Visibility được định nghĩa là tỷ lệ phần trăm tổng diện tích trưng bày của tất cả các sản phẩm trong một cửa hàng được phân bổ cho sản phẩm cụ thể đó. Trong thực tế, nếu một sản phẩm được ghi nhận có doanh số (Item_Outlet_Sales > 0) tại một cửa hàng, sản phẩm đó bắt buộc phải được trưng bày hoặc hiện diện để khách hàng có thể mua. Một sản phẩm có độ hiển thị bằng 0 tuyệt đối nhưng vẫn phát sinh doanh số là điều phi lý trong vận hành bán lẻ truyền thống.

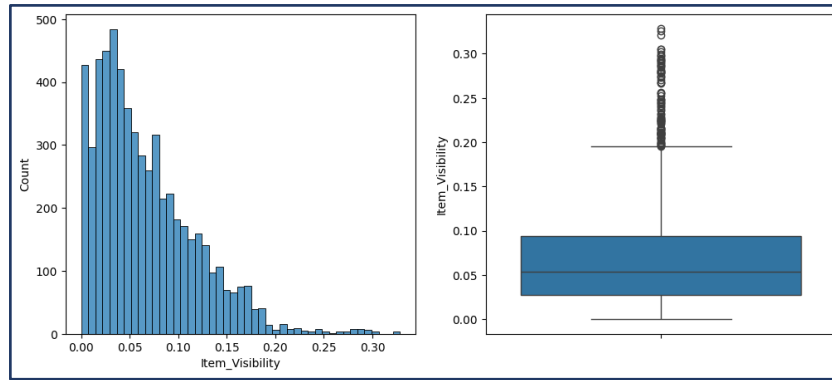
Bước 1: Chuyển giá trị 0 thành NaN (giá trị thiếu)

Theo như tôi đã lập luận ở trên, giá trị 0 tại cột Item_Visibility không phải là thể hiện sản phẩm không trưng bày mà chính là không có dữ liệu, nên cần chuyển thành NaN để dễ dàng xử lý đúng cách, lệnh như sau:

```
X_train_c['Item_Visibility'] = X_train_c['Item_Visibility'].replace(0, np.nan)
```

Bước 2: Tính Item_Visibility trung vị theo từng mã sản phẩm

Biến Item_Visibility được xác định là có phân phối lệch phải và chứa nhiều giá trị ngoại lai (những sản phẩm có diện tích trưng bày cực lớn). Trong thống kê, trung vị là một chỉ số bền vững, không bị kéo lệch bởi các giá trị cực đoan như số trung bình



Hình 3.11. Trực quan hoá phân phối và ngoại lai của biến *Item_Visibility*

Tôi cũng kiểm tra mối tương quan giữa mã sản phẩm (*Item_Identifier*) và diện tích trưng bày của sản phẩm (*Item_Visibility*), kết quả mà tôi nhận được như sau:

	Item_Identifier	Item_Visibility
1245	DRA12	0.040912
1693	DRA12	NaN
8043	DRA12	0.068535
7467	DRA12	0.041113
1148	DRA24	0.040154
...	...	...
4924	NCZ53	0.040970
7926	NCZ54	0.083489
2839	NCZ54	0.083359
1896	NCZ54	0.082956
1969	NCZ54	0.083528

5966 rows x 2 columns

Hình 3.12. Kiểm tra diện tích trưng bày của từng mã sản phẩm

Có thể thấy cùng mã sản phẩm nhưng diện tích trưng bày không hoàn toàn giống nhau, đều có sự dao động nhẹ. Vì vậy, tôi sử dụng giá trị trung vị của từng mã sản phẩm để điền vào các giá trị bị thiếu của chính mã đó. Cách làm này giúp giữ được đặc trưng riêng của từng sản phẩm, đồng thời tránh bị ảnh hưởng bởi các giá trị bất thường.

Để thực hiện, tôi tạo một bảng tổng hợp trung vị theo từng mã sản phẩm:

```
VISIBILITY_ID_AVG_PIVOT =
X_train_c.pivot_table(values='Item_Visibility',
index='Item_Identifier', aggfunc='median').reset_index()
```

Sau đó, tôi chuyển bảng này thành từ điển để máy có thể tra cứu nhanh khi điền dữ liệu:

```
VISIBILITY_ID_MAPPING = dict(zip(VISIBILITY_ID_AVG_PIVOT['Item_Identifier'], VISIBILITY_ID_AVG_PIVOT['Item_Visibility'])) list(VISIBILITY_ID_MAPPING.items())[:10]
```

Kết quả thu được là một bảng từ điển gồm các cặp (Mã sản phẩm – Trung vị diện tích trưng bày), giúp việc điền dữ liệu thiếu được thực hiện nhanh chóng và chính xác hơn.

```
[('DRA12', 0.0456496065), ('DRA24', 0.039990314), ('DRA59', 0.128027378), ('DRB01', 0.082170947), ('DRB13', 0.0070635965), ('DRB24', 0.020651418499999998), ('DRB25', 0.0696496015), ('DRB48', 0.024832806), ('DRC01', 0.0192194725), ('DRC12', 0.037856278)]
```

Hình 3.13. Kết quả từ điển (Mã sản phẩm – Trung vị diện tích sản phẩm)

Bước 3: Tính trung vị hiển thị theo loại sản phẩm

Trong một số trường hợp có những sản phẩm hoàn toàn mới và không có bất kỳ dữ liệu hiển thị nào để có thể tính trung vị theo mã sản phẩm (Item_Identifier). Lúc này tôi cần một cách thay thế hợp lý hơn.

Vì vậy, tôi sử dụng trung vị theo nhóm sản phẩm (Item_Type) để điền giá trị thiếu. Lý do là vì các sản phẩm cùng loại (Food, Drinks, Non-Consumable) thường có cách trưng bày tương đối giống nhau, nên diện tích hiển thị của chúng cũng có xu hướng gần nhau hơn so với các nhóm khác.

	Item_Type	Item_Visibility
6428	Drinks	0.037396
4605	Drinks	0.037827
18	Drinks	0.034238
3146	Drinks	0.045292
4341	Drinks	0.045400
...	...	...
2330	Non-Consumable	NaN
1152	Non-Consumable	0.076156
802	Non-Consumable	NaN
3359	Non-Consumable	0.021413
7270	Non-Consumable	0.016993
5966 rows x 2 columns		

Hình 3.14. Kiểm tra diện tích trung bày của từng loại sản phẩm

Kết quả cho thấy việc sử dụng trung vị theo Item_Type giúp đảm bảo giá trị điền vào hợp lý, ngay cả khi không có thông tin cụ thể của từng sản phẩm.

Để thực hiện, tôi tạo một bảng tổng hợp trung vị theo từng loại sản phẩm:

```
VISIBILITY_TYPE_AVG_PIVOT =
X_train_c.pivot_table(values='Item_Visibility', index='Item_Type',
aggfunc='median').reset_index()
```

Sau đó, tôi chuyển bảng này thành từ điển để máy có thể tra cứu nhanh khi điền dữ liệu:

```
VISIBILITY_TYPE_MAPPING =
dict(zip(VISIBILITY_TYPE_AVG_PIVOT['Item_Type'],
VISIBILITY_TYPE_AVG_PIVOT['Item_Visibility']))
VISIBILITY_TYPE_MAPPING.items()
```

Kết quả thu được là bộ từ điển (Loại sản phẩm – Trung vị diện tích trung bày), giúp việc điền dữ liệu thiếu được thực hiện nhanh chóng và chính xác hơn.

```
dict_items([('Drinks', 0.05653011), ('Food', 0.06173052), ('Non-Consumable',
0.047250965)])
```

Hình 3.15. Kết quả từ điển (Loại sản phẩm – Trung vị diện tích sản phẩm)

Bước 4: Điền các giá trị thiếu từ bảng từ điển

Sau khi đã có hai bảng từ điển (Mã sản phẩm – Trung vị diện tích trung bày) và (Loại sản phẩm – Trung vị diện tích trung bày), tôi xây dựng hàm để điền các giá trị còn thiếu trong cột Item_Visibility.

```
def impute_item_visibility(data_frame):
    data_frame.loc[:, 'Item_Visibility'] = data_frame.loc[:,
    'Item_Visibility'].fillna(
        data_frame.loc[:, 'Item_Identifier'].map(VISIBILITY_ID_MAPPING)
    )
    data_frame.loc[:, 'Item_Visibility'] = data_frame.loc[:,
    'Item_Visibility'].fillna(
        data_frame.loc[:, 'Item_Type'].map(VISIBILITY_TYPE_MAPPING)
    )
    return data_frame
```

Cụ thể, hàm sẽ thực hiện hai bước. Đầu tiên, nó sẽ điền diện tích hiển thị dựa trên mã sản phẩm (Item_Identifier), bằng cách tra cứu từ điển đã tạo. Nếu sản phẩm đã từng xuất hiện trước đó, giá trị Item_Visibility sẽ được khôi phục một cách chính xác hơn.

Nếu sau bước này vẫn còn giá trị bị thiếu, hàm sẽ tiếp tục điền theo nhóm sản phẩm (Item_Type). Lúc này, mỗi sản phẩm sẽ được gán giá trị bằng trung vị diện tích trưng bày của nhóm, như Food, Drinks hoặc Non-Consumable.

Sau khi hoàn tất các bước xử lý, tôi tiến hành kiểm tra lại dữ liệu để đảm bảo không còn giá trị bị thiếu và không còn giá trị bằng 0 không hợp lý trong biến Item_Visibility.

Cụ thể, tôi sử dụng các lệnh kiểm tra số lượng giá trị NaN:

```
X_train_c['Item_Visibility'].isnull().sum()
```

Và đếm lại số giá trị bằng 0 trong cột này:

```
print("Số lượng giá trị bằng 0 còn lại:", (X_train_c['Item_Visibility']
== 0).sum())
```

Kết quả cho thấy dữ liệu đã được xử lý đầy đủ, không còn giá trị bị thiếu và cũng không còn giá trị 0 bất hợp lý.

3.6. Kiểm tra tính hợp lý của dữ liệu

3.6.1. Xử lý tính không hợp lý của biến Item_Fat_Content

Để kiểm tra tính hợp lý của dữ liệu, tôi đã trích các biến dạng phân loại từ tập huấn luyện (train):

```
cat_features = X_train_c.select_dtypes(include='object')
```


Sau đó tôi thực hiện đếm số lần xuất hiện của từng giá trị trong các biến phân loại với hàm `value_counts()`. Kết quả tôi nhận được cho thấy chỉ có biến `Item_Fat_Content` là không có tính hợp lý của dữ liệu:

	count
Item_Fat_Content	
Low Fat	3535
Regular	2027
LF	243
low fat	82
reg	79
dtype:	int64

Hình 3.16. Kết quả biến `Item_Fat_Content` là không có tính hợp lý

Ta có thể thấy các nhãn Low Fat, LF, low fat đều cùng chỉ Low Fat, và Regular, reg đều cùng chỉ Regular. Vì vậy, việc xử lý tính không hợp lý của biến `Item_Fat_Content` bằng cách gom các giá trị cùng ý nghĩa nhưng khác cách viết về cùng một nhãn duy nhất rất đơn giản bằng lệnh:

```
def standardize_item_fat_content(data_frame):
    data_frame['Item_Fat_Content'] =
    data_frame['Item_Fat_Content'].replace({'LF':'Low Fat', 'low
    fat':'Low Fat', 'reg':'Regular'})
    return data_frame
X_train_c = standardize_item_fat_content(X_train_c)
X_train_c['Item_Fat_Content'].value_counts()
```

Cuối cùng tôi được bảng kết quả như sau:

	count
Item_Fat_Content	
Low Fat	3860
Regular	2106
dtype:	int64

Hình 3.17. Kết quả xử lý tính không hợp lý của biến `Item_Fat_Content`

3.6.2. Điều chỉnh hàm lượng chất béo cho sản phẩm không tiêu dùng

Để kiểm tra tính hợp lý của biến `Item_Fat_Content`, tôi thực hiện thống kê số lần xuất hiện của hàm lượng chất béo theo từng loại sản phẩm bằng lệnh:

```
X_train_c.groupby(by=['Item_Type', 'Item_Fat_Content']).size()
```

Kết quả cho thấy có một số điểm chưa hợp lý. Cụ thể, các sản phẩm thuộc nhóm không tiêu dùng (Non-Consumable) vẫn đang được gán các giá trị như Low Fat.

		0
Item_Type	Item_Fat_Content	
Drinks	Low Fat	517
	Regular	43
Food	Low Fat	2206
	Regular	2063
Non-Consumable	Low Fat	1137
		dtype: int64

Hình 3.18. Tần suất `Item_Fat_Content` theo `Item_Type`

Tuy nhiên, trên thực tế các sản phẩm này không phải là thực phẩm nên không thể có hàm lượng chất béo. Điều này cho thấy dữ liệu đang bị sai về mặt logic và có thể gây sai lệch cho mô hình khi học dữ liệu.

Để kiểm tra rõ hơn, tôi lọc riêng các sản phẩm không tiêu dùng bằng lệnh:

```
X_train_c.loc[X_train_c['Item_Type'] == 'Non-Consumable',  
              'Item_Fat_Content']
```

Kết quả mà tôi nhận được không khác ngoài dự đoán, toàn bộ sản phẩm không tiêu dùng đều bị gán nhãn hàm lượng chất béo là Low Fat:

Item_Fat_Content	
655	Low Fat
4423	Low Fat
907	Low Fat
7021	Low Fat
6074	Low Fat
...	...
8322	Low Fat
466	Low Fat
6265	Low Fat
5390	Low Fat
7270	Low Fat
1137 rows x 1 columns	
dtype: object	

Hình 3.19. Hàm lượng chất béo của sản phẩm Non-Consumable

Để xử lý nhãn bị gán sai logic này tôi tiến hành gán lại giá trị phù hợp hơn cho nhóm sản phẩm không tiêu dùng. Cụ thể, tôi thay toàn bộ giá trị Item_Fat_Content của nhóm này thành "Non_Edible" để phản ánh đúng bản chất của sản phẩm.

```
def correct_item_fat_content(data_frame):
    data_frame.loc[data_frame['Item_Type'] == 'Non-Consumable',
                    'Item_Fat_Content'] = 'Non_Edible'
    return data_frame

X_train_c = correct_item_fat_content(X_train_c)
X_train_c.groupby(by=['Item_Type', 'Item_Fat_Content']).size()
```

Cuối cùng, dữ liệu đã trở nên hợp lý hơn khi các sản phẩm không tiêu dùng không còn bị gán các giá trị liên quan đến hàm lượng chất béo.

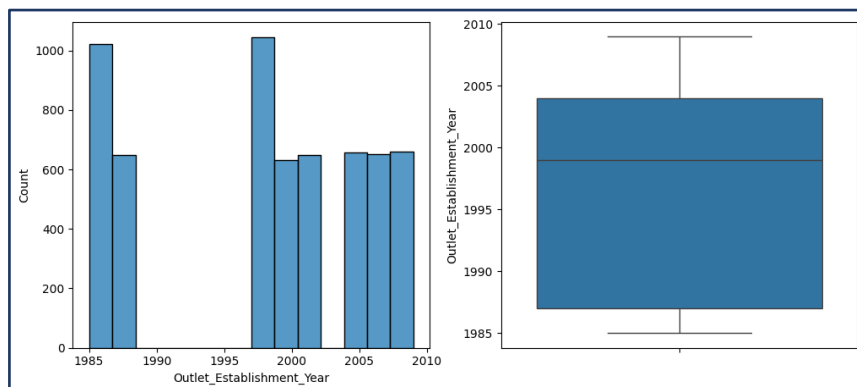
0		
Item_Type	Item_Fat_Content	
Drinks	Low Fat	517
	Regular	43
Food	Low Fat	2206
	Regular	2063
Non-Consumable	Non_Edible	1137

dtype: int64

Hình 3.20. Kết quả điều chỉnh Item_Fat_Content cho Non-Consumable

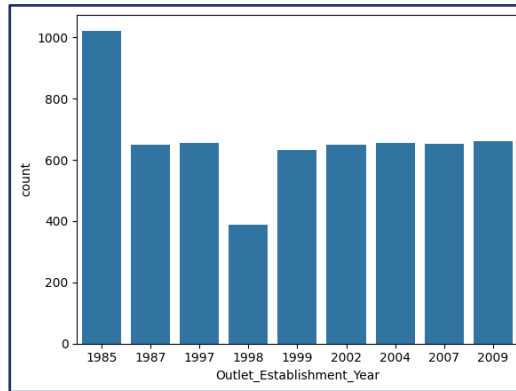
3.7. Xử lý biến thời gian và tạo biến mới

Trong dữ liệu ban đầu, biến Outlet_Establishment_Year chỉ đơn giản là năm thành lập của cửa hàng. Tuy nhiên, nếu giữ nguyên biến này, mô hình sẽ phải học từ các con số năm rời rạc, điều này không phản ánh rõ ý nghĩa thực tế. Để đánh giá đặc điểm của biến này, tôi tiến hành trực quan hóa dữ liệu qua hai phương pháp:



Hình 3.21. Trực quan hóa phân phối và ngoại lai của biến Outlet_Establishment_Year

Biểu đồ Boxplot xác nhận biến này không có giá trị ngoại lai, dữ liệu nằm gọn trong khoảng từ năm 1985 đến 2009. Biểu đồ Histogram cho thấy dữ liệu tập trung vào một số mốc năm nhất định thay vì phân bố đều, cho thấy các cửa hàng được mở theo từng giai đoạn cụ thể.



Hình 3.22. Tần suất phân bố của biến *Outlet_Establishment_Year*

Biểu đồ này giúp quan sát tần suất của các sản phẩm được bán ứng với từng năm thành lập cửa hàng khác nhau. Biểu đồ sẽ cho thấy số lượng bản ghi tập trung tần suất bán hàng cao nhất tại các cửa hàng thành lập năm 1985 và thấp nhất tại năm 1998. Trong thực tế, các cửa hàng hoạt động lâu năm thường có khách hàng quen, quy trình ổn định, nên có khả năng đạt doanh số tốt hơn. Vì vậy, “tuổi cửa hàng” là một yếu tố có liên quan trực tiếp đến doanh số, hợp lý hơn so với việc dùng năm thành lập riêng lẻ.

Vì vậy, tôi thực hiện tạo một biến mới là *Outlet_Age* (tuổi cửa hàng) bằng cách lấy năm dữ liệu (2013) trừ đi năm thành lập. Biến này thể hiện số năm cửa hàng đã hoạt động, tức là thâm niên của cửa hàng trên thị trường và đồng thời xóa biến *Outlet_Establishment_Year* nhằm tránh bị nhiễu dữ liệu khi chạy mô hình.

```
def create_outlet_age(data_frame):
    data_frame['Outlet_Age'] = 2013 -
        data_frame['Outlet_Establishment_Year']
    data_frame = data_frame.drop(columns=['Outlet_Establishment_Year'])
    return data_frame

X_train_c = create_outlet_age(X_train_c)
```

3.8. Loại bỏ các biến không cần thiết

Sau khi thực hiện các bước tiền xử lý, một số biến không cần thiết hoặc không đóng góp nhiều vào việc dự báo được loại bỏ khỏi tập dữ liệu. Các biến định danh như *Item_Identifier* và *Outlet_Identifier* thường không mang ý nghĩa phân tích và có thể làm tăng độ phức tạp của mô hình mà không cải thiện hiệu quả dự báo.

```
cat_features = X_train.select_dtypes(include='object')
```

```
cat_features = cat_features.drop(['Item_Identifier',  
'Outlet_Identifier'], axis=1)  
cat_features.head()
```

Phải loại bỏ hai biến này trước khi chạy mô hình vì biến `Item_Identifier` có tới 1.559 giá trị duy nhất (1.543 giá trị duy nhất trong tập huấn luyện). Nếu sử dụng phương pháp One-Hot Encoding (tách mỗi mã thành một cột riêng), bộ dữ liệu sẽ đột ngột tăng thêm hơn 1.500 cột mới. Điều này làm cho ma trận dữ liệu trở nên cực kỳ thưa thớt, khiến mô hình trở nên nặng nề, tốn rất nhiều tài nguyên tính toán và thời gian huấn luyện.

Mặc dù biến `Outlet_Identifier` chỉ có 10 giá trị (tương ứng với 10 cửa hàng) và không quá phức tạp như `Item_Identifier`, nhưng trong bài này tôi quyết định loại bỏ biến này khỏi mô hình vì `Outlet_Identifier` chỉ là mã cửa hàng không mang ý nghĩa về đặc điểm hay hoạt động. Trong khi đó, dữ liệu đã có các biến quan trọng hơn như `Outlet_Type`, `Outlet_Size`, `Outlet_Location_Type` để mô tả cửa hàng, nên giữ thêm biến này sẽ bị trùng thông tin.

3.9. Chuẩn bị dữ liệu cho mô hình

Chuẩn bị dữ liệu là bước rất quan trọng trong bài toán Machine Learning, đây là quá trình biến dữ liệu thô thành dữ liệu “sạch” đã được xử lý và dễ hiểu hơn để mô hình có thể học và dự đoán tốt hơn.

3.9.1. Áp dụng các bước xử lý dữ liệu cho tập train – test

Thay vì làm từng bước riêng lẻ, tôi gom tất cả các bước xử lý dữ liệu vào một hàm tên là `prepare_data`. Cách làm này giúp: Dữ liệu luôn được xử lý giống nhau (không bị lệch giữa các lần chạy), tránh sai sót khi làm thủ công, dễ áp dụng lại cho dữ liệu mới. Hiểu đơn giản, hàm này giống như một quy trình cố định, cứ đưa dữ liệu vào là sẽ được xử lý đầy đủ.

Bước 1: Tạo hàm `prepare_data`

```
def prepare_data(data_frame):  
    data_frame = create_item_type_column(data_frame)  
    data_frame = impute_item_weight(data_frame)  
    data_frame = impute_outlet_size(data_frame)  
    data_frame = standardize_item_fat_content(data_frame)  
    data_frame = correct_item_fat_content(data_frame)
```



```
data_frame = impute_item_visibility(data_frame)
data_frame = create_outlet_age(data_frame)
return data_frame
```

Bước 2: Áp dụng cho tập train

Tôi áp dụng hàm này cho tập train để đảm bảo dữ liệu dùng để huấn luyện đã được làm sạch hoàn toàn:

```
X_train = prepare_data(X_train)
X_train.isnull().sum()
```

Bước 3: Áp dụng cho tập test

Sau đó, tôi tiếp tục áp dụng cùng hàm này cho tập test:

```
X_test = prepare_data(X_test)
X_test.isnull().sum()
```

Việc này giúp đảm bảo dữ liệu test cũng được xử lý giống hệt như train. Nhờ đó, mô hình sẽ dự đoán chính xác hơn khi gặp dữ liệu mới.

3.9.2. Xử lý biến phân loại (biến định tính) để mô hình hiểu

Hầu hết các thuật toán học máy không thể làm việc trực tiếp với dữ liệu dạng chữ. Do đó, chúng ta cần chuyển đổi các biến định tính thành dạng số để mô hình có thể thực hiện các phép toán đại số.

Thay vì dùng số thứ tự (Integer Encoding), chúng ta sử dụng phương pháp One-Hot Encoding (OHE). Phương pháp này tạo ra các cột Boolean (0 và 1) cho mỗi giá trị duy nhất của biến, giúp mô hình không hiểu lầm rằng các hạng mục có một thứ tự ưu tiên nào đó.

Bước 1: Trích xuất biến định tính và loại bỏ mã định danh

```
cat_features = X_train.select_dtypes(include='object')
cat_features = cat_features.drop(['Item_Identifier',
'Outlet_Identifier'], axis=1)
cat_features.head()
```

Bước 2: Tạo bộ mã hoá

Việc tôi thiết lập `handle_unknown='ignore'` cực kỳ quan trọng. Nó giúp mô hình hoạt động ổn định nếu sau này gặp một nhãn mới trong tập test hoặc dữ liệu thực tế mà tập train chưa từng thấy, khi đó mô hình sẽ bỏ qua thay vì báo lỗi. Lưu ý rằng ta chỉ học các nhãn trên tập train để tránh rò rỉ dữ liệu.

```
ohe = OneHotEncoder(handle_unknown='ignore')
```

```
ohe.fit(cat_features)
```

Bước 3: Lấy tên các cột sau khi chuyển đổi

Lấy tên các cột sau khi chuyển đổi giúp chúng ta giữ được tính minh bạch của dữ liệu, biết chính xác cột nào đại diện cho nhãn nào để phục vụ việc phân tích độ quan trọng của biến sau này.

```
ohe_feature_names =  
ohe.get_feature_names_out(input_features=cat_features.columns)
```

Bước 4: Giữ nguyên các biến định lượng

Các biến số như Item_MRP, Item_Weight hay biến mới Outlet_Age được tách riêng và reset index để đảm bảo khi nối lại với các cột định tính sau mã hóa, các dòng dữ liệu sẽ khớp nhau hoàn toàn.

```
num_feats_train =  
X_train.select_dtypes(exclude=['object']).reset_index(drop=True)
```

Bước 5: Chuyển các biến định tính thành số 0, 1 và tổng hợp dữ liệu cuối cùng

Sau khi chuyển đổi tập train thành dạng số hoàn toàn thông qua transform, chúng ta nối phần số và phần định tính đã mã hóa để tạo thành tập dữ liệu huấn luyện cuối cùng (X_train_final).

```
cat_features_train = X_train.select_dtypes(include=['object'])  
cat_features_train = cat_features_train.drop(['Item_Identifier',  
'Outlet_Identifier'], axis=1)  
X_train_cat_ohe =  
pd.DataFrame(ohe.transform(cat_features_train).toarray(),  
columns=ohe_feature_names)  
X_train_final = pd.concat([num_feats_train, X_train_cat_ohe], axis=1)
```

Để mô hình có thể dự báo, tập test phải có cấu trúc, số lượng cột và thứ tự cột y hệt tập train. Việc tôi dùng lại chính đối tượng ohe đã học từ tập train để transform tập test đảm bảo tính nhất quán tuyệt đối trong quy trình xử lý dữ liệu.

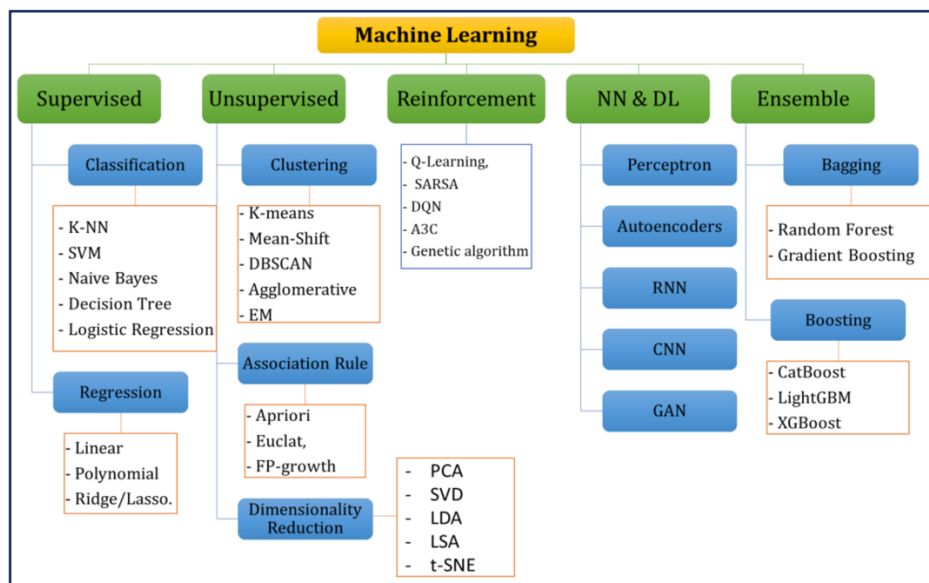
```
num_feats_train =  
X_test.select_dtypes(exclude=['object']).reset_index(drop=True)  
cat_feats_test = cat_feats_test.drop(['Item_Identifier',  
'Outlet_Identifier'], axis=1)  
X_test_cat_ohe = pd.DataFrame(ohe.transform(cat_feats_test).toarray(),  
columns=ohe_feature_names)
```

```
X_test_final = pd.concat([num_feats_train, X_test_cat_ohc], axis=1)
X_test_final = X_test_final[final_columns]
```

PHẦN IV. MÔ HÌNH HOÁ DỮ LIỆU

4.1. Cơ sở lý thuyết mô hình

Học máy là một nhánh quan trọng của khoa học dữ liệu, tập trung vào việc xây dựng các mô hình có khả năng học từ dữ liệu để đưa ra dự đoán hoặc quyết định mà không cần lập trình tường minh. Theo Ali et al. (2023), quy trình học máy bao gồm các bước chính như thu thập dữ liệu, tiền xử lý, phân tích khám phá, xây dựng mô hình, đánh giá và triển khai. Trong nghiên cứu này, một số mô hình tiêu biểu được sử dụng bao gồm Linear Regression, Ridge & Lasso Regression, Random Forest và XGBoost.



Hình 4.1. Sơ đồ về Machine Learning (Ali et al., 2023)

4.1.1. Linear Regression

Hồi quy tuyến tính là một trong những phương pháp đơn giản và phổ biến nhất trong học máy có giám sát dùng để dự đoán một giá trị định lượng cụ thể. Mục tiêu của nó là mô hình hóa mối quan hệ giữa một hoặc nhiều biến đầu vào và một biến mục tiêu liên tục.

Mô hình này dựa trên giả định rằng có một mối quan hệ tuyến tính giữa các biến đầu vào (X) và đầu ra (Y).

Mô hình hồi quy tuyến tính có hai dạng chính:

- Hồi quy tuyến tính đơn biến: Gồm n quan sát có biến phụ thuộc là $y = \{y_1, y_2, \dots, y_n\}$ và biến đầu vào $x = \{x_1, x_2, \dots, x_n\}$. Véc tơ $w = (w_0, w_1)$ có giá trị

w_0, w_1 lần lượt là hệ số góc và hệ số ước lượng. Phương trình hồi qui tuyến tính đơn biến có dạng:

$$\hat{y}_i = f(x_i) = w_0 + w_1 x$$

- Hồi qui tuyến tính đa biến: là hồi qui tuyến tính với nhiều hơn một biến đầu vào. Hồi qui tuyến tính đa biến phổ biến hơn so với đơn biến vì trên thực tế rất hiếm các tác vụ dự báo chỉ gồm một biến đầu vào. Phương trình hồi qui của nó có dạng:

$$\hat{y}_i = f(x_1, x_2, \dots, x_p) = w_0 + w_1 x_{i1} + \dots + w_p x_{ip} = w^T x_i$$

4.1.2. Lasso & Ridge Regression

Trong quá trình huấn luyện các mô hình học máy, quá khớp là hiện tượng mô hình khớp rất tốt trên tập dữ liệu huấn luyện nhưng lại dự báo kém trên dữ liệu kiểm tra. Hiện tượng này khiến mô hình mất đi tính đại diện và không thể áp dụng vào thực tiễn do các dự báo bị sai lệch. Nguyên nhân có thể do phân phối giữa tập huấn luyện và dự báo khác xa nhau, hoặc do mô hình quá phức tạp với quá nhiều tham số.

Để khắc phục hiện tượng quá khớp, kỹ thuật hiệu chuẩn được áp dụng bằng cách cộng thêm vào hàm mất mát một thành phần hiệu chuẩn. Thành phần này thường ở dạng chuẩn bậc 1 hoặc bậc 2 của các hệ số hồi qui nhằm kiểm soát độ lớn của các tham số.

Ridge Regression sử dụng thành phần hiệu chuẩn là chuẩn bậc 2 của các hệ số. Hàm mất mát của Ridge Regression có dạng:

$$\mathcal{L}(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \|w\|_2^2$$

Khác với Ridge, Lasso Regression sử dụng chuẩn bậc 1 làm thành phần hiệu chuẩn. Phương trình hàm mất mát được biểu diễn như sau:

$$\mathcal{L}(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \|w\|_1$$

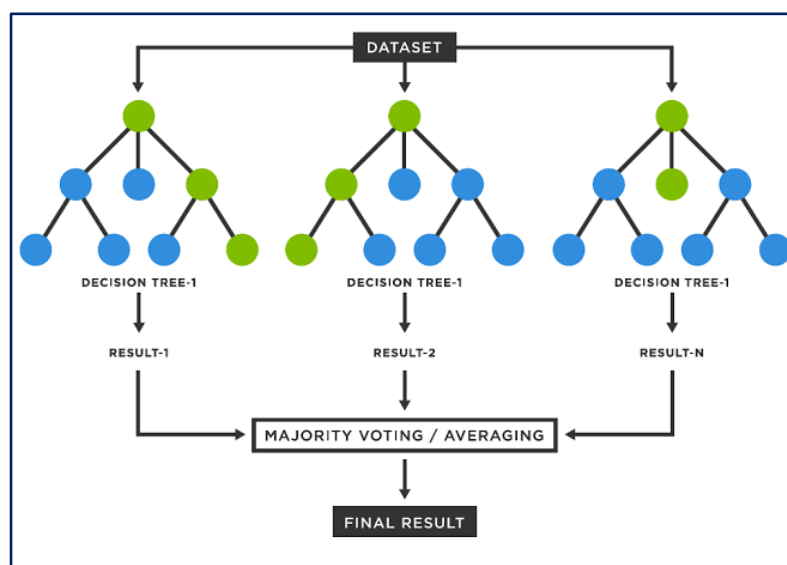
Đặc điểm quan trọng của Lasso là nó có khả năng triệt tiêu các hệ số của các biến ít quan trọng về mức 0, từ đó đóng vai trò như một kỹ thuật lựa chọn biến tự động.

Đối với những hồi qui này thì chúng ta cần tinh chỉnh hệ số α để tìm ra một hệ số là tốt nhất với từng bộ dữ liệu:

- Trong trường hợp dữ liệu bị quá khớp nặng thì cần giảm quá khớp bằng cách gia tăng ảnh hưởng của thành phần điều chuẩn thông qua tăng hệ số α .
- Nếu mô hình không bị quá khớp thì có thể lựa chọn α gần 0.
- Trường hợp $\alpha=0$ thì phương trình hồi qui tương đương với hồi qui tuyến tính đa biến.

4.1.3. Random Forest

Random Forest là một phương pháp học máy kết hợp dựa trên các cây quyết định, thuật toán này được phát triển bởi Leo Breiman và Adele Cutler vào năm 2001. Bản chất của Random Forest là sự kết hợp giữa kỹ thuật Bagging (Bootstrap Aggregating) và việc lựa chọn các đặc trưng ngẫu nhiên để xây dựng một tập hợp các cây quyết định độc lập, sau đó tổng hợp kết quả của chúng để đưa ra dự báo cuối cùng.



Hình 4.2. Ví dụ về mô hình RandomForest

Random Forest là một bộ phân loại bao gồm một tập hợp các cấu trúc cây dự đoán $\{h(x, \theta_k), k = 1, \dots\}$, trong đó các $\{\theta_k\}$ là các vector ngẫu nhiên độc lập và có cùng phân phối (i.i.d.). Mỗi cây sẽ bỏ một phiếu đơn vị cho lớp phổ biến nhất tại đầu vào x .

Một đặc điểm quan trọng của Random Forest là nó không bị quá khớp khi số lượng cây tăng lên. Thay vào đó, sai số tổng quát hóa (PE^*) sẽ hội tụ về một giới hạn nhờ vào định luật số lớn.

Sự hội tụ và vấn đề quá khớp

Một đặc điểm quan trọng của Random Forest là khả năng không bị quá khớp khi số lượng cây tăng lên. Theo Leo Breiman, khi số lượng cây ($k \rightarrow \infty$), sai số tổng quát hóa hội tụ hầu khắp nơi về:

$$P_{X,Y}(P_{\Theta}(h(X, \Theta) = Y) - \max_{j \neq Y} P_{\Theta}(h(X, \Theta) = j) < 0)$$

Hàm lẻ

Để phân tích hiệu suất mô hình, Breiman đưa ra khái niệm hàm lẻ. Hàm lẻ đo lường mức độ mà số phiếu bầu trung bình cho lớp đúng (Y) vượt quá số phiếu bầu trung bình cho bất kỳ lớp nào khác. Giá trị lẻ càng lớn thì độ tin cậy của phân loại càng cao. Công thức:

$$mr(X, Y) = P_{\Theta}(h(X, \Theta) = Y) - \max_{j \neq Y} P_{\Theta}(h(X, \Theta) = j)$$

Độ mạnh và Sự tương quan

Sai số tổng quát hóa của rừng phụ thuộc vào hai yếu tố chính: độ mạnh của các cây đơn lẻ và sự tương quan giữa chúng.

- **Độ mạnh (s):** Là giá trị kỳ vọng của hàm lẻ: $s = E_{X,Y}mr(X, Y)$.
- **Sự tương quan ($\bar{\rho}$):** Là giá trị tương quan trung bình giữa các hàm lẻ thô của các cặp cây khác nhau trong rừng.

Breiman đã chứng minh một giới hạn trên cho sai số tổng quát hóa dựa trên độ mạnh và sự tương quan:

$$PE^* \leq \bar{p} \frac{(1 - s^2)}{s^2}$$

Ý nghĩa: Để giảm sai số, mục tiêu là tối ưu hóa tỷ lệ $\bar{p}/s^2$ – nghĩa là giảm sự tương quan giữa các cây trong khi vẫn duy trì độ mạnh của chúng.

Cơ chế tạo sự ngẫu nhiên

Để giảm sự tương quan ($\bar{\rho}$), Random Forest sử dụng các phương pháp tiêm vào sự ngẫu nhiên:

- **Bagging:** Mỗi cây được huấn luyện trên một tập dữ liệu con được lấy mẫu ngẫu nhiên có hoàn lại (bootstrap sample) từ tập gốc.
- **Forest-RI (Random Input Selection):** Tại mỗi nút, một nhóm biến đầu vào (F) được chọn ngẫu nhiên để tìm điểm phân tách tốt nhất.
- **Forest-RC (Random Combinations):** Tạo ra các đặc trưng mới bằng cách kết hợp tuyến tính ngẫu nhiên các biến đầu vào hiện có.

Ước lượng Out-of-bag (OOB)

Khoảng 1/3 dữ liệu không được sử dụng trong mỗi lần bootstrap (gọi là dữ liệu out-of-bag). Dữ liệu này được dùng để:

- Ước lượng sai số tổng quát hóa mà không cần bộ dữ liệu kiểm tra riêng biệt.
- Đo lường độ mạnh và sự tương quan của rừng.
- Đánh giá độ quan trọng của biến: Bằng cách xáo trộn ngẫu nhiên giá trị của một biến trong dữ liệu OOB và đo lường mức độ tăng của sai số phân loại.

Random Forest cho hồi quy

Trong hồi quy, kết quả đầu ra là các giá trị số và hàm dự báo của rừng là trung bình cộng của các cây $\{h(x, \theta_k)\}$

- **Sai số tổng quát hóa (MSE):** $PE * (forest) = E_{X,Y}(Y - E_{\theta}h(X, \theta))^2$.
- **Công thức giới hạn sai số:** $PE * (forest) \leq \bar{\rho}PE^*(tree)$

Trong đó, $\bar{\rho}$ là sự tương quan giữa các phần dư $Y - h(X, \theta)$ và $Y - h(X, \theta')$. Mô hình hồi quy rừng hiệu quả khi sự tương quan giữa các phần dư thấp và các cây có sai số thấp.

Về mặt trực quan, Random Forest có thể được hiểu như một hệ thống gồm nhiều cây quyết định hoạt động song song, trong đó mỗi cây được huấn luyện trên các tập dữ liệu và tập đặc trưng khác nhau. Cụ thể, mỗi cây được xây dựng từ một mẫu bootstrap của dữ liệu gốc và tại mỗi nút phân tách chỉ xem xét một tập con ngẫu nhiên các biến đầu vào. Nhờ đó, các cây trong rừng có sự đa dạng về cấu trúc và cách học, góp phần làm giảm tương quan giữa các mô hình thành phần – yếu tố then chốt giúp cải thiện sai số tổng quát hóa như đã phân tích ở phần lý thuyết.

Quá trình dự đoán của Random Forest được thực hiện bằng cách tổng hợp kết quả từ tất cả các cây. Đối với bài toán phân loại, mỗi cây sẽ đưa ra một nhãn dự

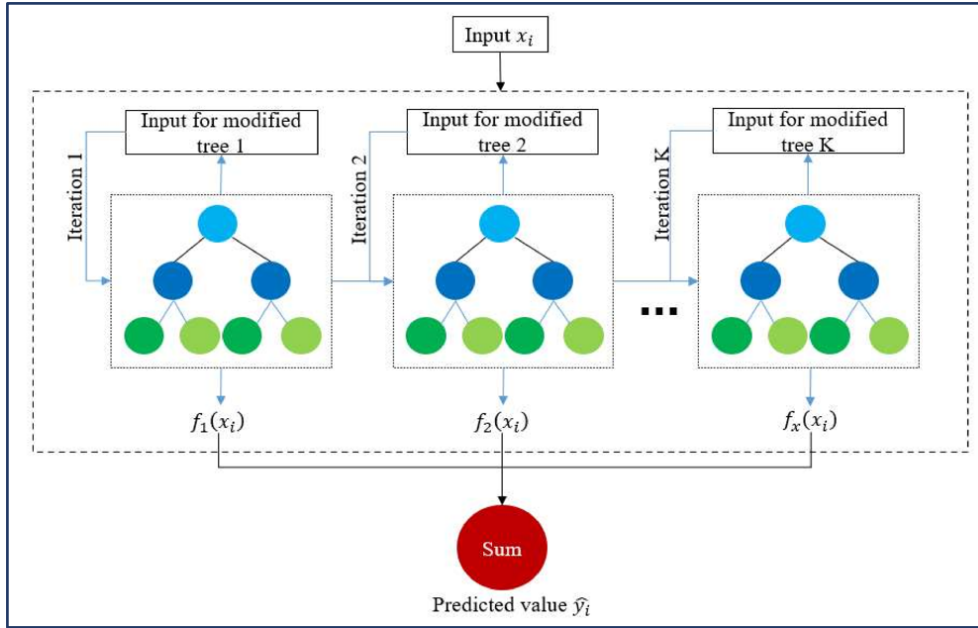
đoán và kết quả cuối cùng được xác định theo nguyên tắc bỏ phiếu đa số. Trong khi đó, đối với bài toán hồi quy, đầu ra của mô hình là trung bình cộng các giá trị dự báo từ các cây. Cơ chế này giúp mô hình tận dụng được “trí tuệ tập thể”, tương tự như việc tổng hợp ý kiến từ nhiều chuyên gia độc lập để đưa ra quyết định chính xác hơn.

Trong thực tế triển khai, Random Forest có ưu điểm là ít yêu cầu tiền xử lý dữ liệu so với nhiều mô hình khác, do các cây quyết định có khả năng xử lý tốt dữ liệu không chuẩn hóa và ít nhạy cảm với ngoại lai. Tuy nhiên, các bước cơ bản như mã hóa biến phân loại và xử lý dữ liệu thiếu vẫn cần được thực hiện để đảm bảo chất lượng đầu vào. Dữ liệu sau đó thường được chia thành tập huấn luyện và tập kiểm tra nhằm đánh giá khách quan hiệu suất mô hình.

Quá trình huấn luyện được thực hiện bằng cách xây dựng nhiều cây quyết định thông qua các thư viện học máy (chẳng hạn như RandomForestRegressor hoặc RandomForestClassifier). Để tối ưu hiệu quả dự báo, các siêu tham số quan trọng như số lượng cây (n_estimators) và độ sâu tối đa của cây (max_depth) thường được điều chỉnh bằng các phương pháp tìm kiếm như RandomizedSearchCV. Việc lựa chọn phù hợp các tham số này giúp cân bằng giữa độ chính xác, khả năng tổng quát hóa và chi phí tính toán của mô hình.

4.1.4. XGBoost

XGBoost (eXtreme Gradient Boosting) là một thư viện học máy mã nguồn mở cung cấp thuật toán thúc đẩy gradient (gradient boosting) hiệu quả, linh hoạt và có khả năng mở rộng cao. Được phát triển bởi Tianqi Chen, XGBoost dựa trên khung thuật toán của phương pháp thúc đẩy gradient truyền thống nhưng được cải tiến mạnh mẽ về mặt toán học và tối ưu hệ thống.



Hình 4.3. Ví dụ về mô hình XGBoost

XGBoost tích hợp trực tiếp các thành phần hiệu chuẩn vào hàm mất mát để kiểm soát độ phức tạp của mô hình. Thành phần này bao gồm hình phạt cho số lượng lá cây và độ lớn của các trọng số tại mỗi lá, giúp ngăn chặn hiện tượng quá khớp hiệu quả hơn. Thuật toán có cơ chế nhận biết độ thừa, tự động tìm hướng phân tách tối ưu cho các điểm dữ liệu bị thiếu hoặc có giá trị bằng không thay vì yêu cầu xử lý dữ liệu trước. XGBoost sử dụng cấu trúc dữ liệu DMatrix được tối ưu hóa để tăng tốc độ huấn luyện và tiết kiệm bộ nhớ. Nó hỗ trợ tính toán song song và phân tán, cho phép xử lý các tập dữ liệu khổng lồ vượt quá bộ nhớ RAM.

Mô hình Ensemble Cây (Tree Ensemble Model)

XGBoost sử dụng mô hình ensemble cây, dự đoán đầu ra bằng cách cộng dồn kết quả từ K hàm (cây quyết định). Công thức dự đoán:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), f_k \in \mathcal{F}$$

Trong đó, $\mathcal{F}$ là không gian của các cây hồi quy (CART). Mỗi cây f_k tương ứng với một cấu trúc cây q độc lập và trọng số lá w . Cấu trúc q ánh xạ một mẫu dữ liệu tới chỉ số lá tương ứng, và w_i là điểm số trên lá thứ i .

Hàm mục tiêu hiệu chuẩn

Để tìm tập hợp các hàm sử dụng trong mô hình, XGBoost tối thiểu hóa hàm mục tiêu có chứa thành phần hiệu chuẩn nhằm tránh hiện tượng quá khớp (overfitting).

- **Hàm mục tiêu tổng quát:**

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

- **Thành phần hiệu chuẩn Ω :**

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

Trong đó, T là số lượng lá trong cây. Hàm l là hàm mất mát lỗi, vi phân được, đo lường sự khác biệt giữa dự đoán $\hat{y}_i$ và mục tiêu y_i . Việc bổ sung Ω giúp làm mượt các trọng số học được để tăng khả năng tổng quát hóa.

Huấn luyện thúc đẩy Gradient

Mô hình được huấn luyện theo kiểu cộng dồn. Tại bước thứ t , ta thêm hàm f_t để tối thiểu hóa hàm mục tiêu.

- **Xấp xỉ Taylor bậc hai:** Để tối ưu hóa nhanh chóng trong bối cảnh tổng quát, thuật toán sử dụng xấp xỉ bậc hai:

$$\mathcal{L}^{(t)} \cong \sum_{i=1}^n [l(y_i, \hat{y}^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

- Với $g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ (gradient bậc một) và $h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}^{(t-1)})$ (gradient bậc hai).

Hàm mục tiêu rút gọn tại bước t

$$\tilde{\mathcal{L}}^{(t)} \cong \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

Điểm số cấu trúc và Tìm kiếm phân tách

- **Trọng số lá tối ưu:** Với một cấu trúc cây q cố định, trọng số tối ưu w_j^* của lá j được tính bằng:

$$w_j^* = \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

- **Điểm số chất lượng cấu trúc:** Giá trị tối ưu tương ứng của hàm mục tiêu dùng để đánh giá chất lượng cây là:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

- **Công thức tính mức giảm tổn thất (Gain) khi phân tách:**

$$\mathcal{L}_{split} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

Công thức này đánh giá các ứng viên phân tách dựa trên sự thay đổi điểm số cấu trúc.

Để đạt được hiệu suất tối ưu, mô hình XGBoost sử dụng nhiều tham số và kỹ thuật điều chỉnh quan trọng. Trước hết, tốc độ học đóng vai trò như một hệ số thu nhỏ áp dụng cho mỗi cây mới, trong đó giá trị nhỏ giúp mô hình học chậm hơn nhưng ổn định hơn và cải thiện khả năng tổng quát hóa trên dữ liệu chưa thấy. Bên cạnh đó, kỹ thuật dừng sớm cho phép quá trình huấn luyện kết thúc khi hiệu suất trên tập kiểm chứng không còn được cải thiện sau một số vòng lặp nhất định, từ đó giúp tiết kiệm thời gian và hạn chế hiện tượng quá khớp. Ngoài ra, XGBoost còn áp dụng lấy mẫu ngẫu nhiên theo cả hàng và cột, giúp tăng tính đa dạng giữa các cây và giảm sự tương quan trong mô hình ensemble. Cuối cùng, tham số gamma kiểm soát mức giảm tổn thất tối thiểu cần thiết để thực hiện một lần phân tách nút mới; giá trị gamma càng lớn thì mô hình càng đơn giản và ít có xu hướng overfitting.

4.2. Triển khai mô hình trên dữ liệu

Trong nghiên cứu này, các mô hình học máy được triển khai bằng ngôn ngữ Python với sự hỗ trợ của các thư viện phổ biến như scikit-learn và xgboost. Trước hết, các thư viện cần thiết được import nhằm phục vụ cho quá trình xây dựng, huấn luyện và đánh giá mô hình.

```
from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
```

```

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Lasso
from xgboost import XGBRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import numpy as np

```

Tôi xây dựng một hàm đánh giá tổng hợp mang tên OS_ML được xây dựng để tính toán các chỉ số quan trọng như MAE, RMSE, R^2 và Bias trên cả tập huấn luyện (train) và tập kiểm tra (test). Hàm này giúp chuẩn hóa việc so sánh hiệu suất giữa các mô hình một cách nhất quán.

```

def OS_ML(Y_train, Y_train_pred, Y_test, Y_test_pred, name=''):
    df_res = pd.DataFrame(columns = ['MAE', 'RMSE', 'R2', 'Bias'],
        index=['Train', 'Test'])
    df_res.index.name = name
    # Tính toán cho tập Train
    df_res.loc['Train', 'MAE'] = mean_absolute_error(Y_train,
        Y_train_pred)
    df_res.loc['Train', 'RMSE'] = np.sqrt(mean_squared_error(Y_train,
        Y_train_pred))
    df_res.loc['Train', 'R2'] = r2_score(Y_train, Y_train_pred)
    df_res.loc['Train', 'Bias'] = np.mean(Y_train - Y_train_pred)
    # Tính toán cho tập Test
    df_res.loc['Test', 'MAE'] = mean_absolute_error(Y_test, Y_test_pred)
    df_res.loc['Test', 'RMSE'] = np.sqrt(mean_squared_error(Y_test,
        Y_test_pred))
    df_res.loc['Test', 'R2'] = r2_score(Y_test, Y_test_pred)
    df_res.loc['Test', 'Bias'] = np.mean(Y_test - Y_test_pred)
    return df_res.astype(float).round(2)

```

Đầu tiên tôi chạy mô hình hồi quy tuyến tính (Linear Regression) được triển khai như một mô hình cơ sở. Mô hình được huấn luyện bằng phương thức `.fit()` trên tập huấn luyện và sau đó được sử dụng để dự đoán trên cả tập train và test. Kết quả dự báo được đưa vào hàm OS_ML để đánh giá, từ đó làm cơ sở so sánh với các mô hình phức tạp hơn.


```

lr = LinearRegression()
lr.fit(X_train_final, y_train)
results_lr = OS_ML(y_train, lr.predict(X_train_final), y_test,
lr.predict(X_test_final), name='Linear Regression')
print(results_lr)

```

Tiếp đến, tôi chạy các mô hình Ridge và Lasso Regression, trong bài nghiên cứu này tôi áp dụng kỹ thuật GridSearchCV để tìm kiếm giá trị tối ưu của tham số điều chuẩn α . Cụ thể, một tập các giá trị α được định nghĩa trước, sau đó mô hình được huấn luyện và đánh giá thông qua phương pháp cross-validation với 5 folds. Việc sử dụng GridSearchCV giúp lựa chọn được mô hình có sai số nhỏ nhất trên dữ liệu kiểm chứng, từ đó cải thiện khả năng tổng quát hóa. Sau khi tìm được tham số tối ưu, mô hình tiếp tục được sử dụng để dự đoán và đánh giá tương tự như Linear Regression.

```

ridge = Ridge(random_state=42)
ridge_params = {'alpha': [0.1, 1, 10, 100]}
ridge_cv = GridSearchCV(
    ridge,
    ridge_params,
    cv=5,
    scoring='neg_root_mean_squared_error'
).fit(X_train_final, y_train)
results_ridge = OS_ML(y_train, ridge_cv.predict(X_train_final), y_test,
ridge_cv.predict(X_test_final), name='Ridge')
print(results_ridge)

```

```

lasso = Lasso(random_state=42, max_iter=100000)
lasso_params = {'alpha': [0.1, 1, 10, 100]}
lasso_cv = GridSearchCV(
    lasso,
    lasso_params,
    cv=5,
    scoring='neg_root_mean_squared_error'
).fit(X_train_final, y_train)
results_lasso = OS_ML(y_train, lasso_cv.predict(X_train_final), y_test,
lasso_cv.predict(X_test_final), name='Lasso')
print(results_lasso)

```

Mô hình XGBoost sẽ là mô hình lựa chọn tiếp theo để tôi chạy, một quy trình tối ưu hóa tham số được thực hiện thông qua GridSearchCV với các siêu tham số quan trọng như số lượng cây (n_estimators), độ sâu cây (max_depth), tốc độ học (learning_rate) và tỷ lệ lấy mẫu (subsample). Mô hình được đánh giá bằng chỉ số RMSE thông qua cross-validation 3 folds. Sau khi xác định được bộ tham số tốt nhất, mô hình XGBoost tối ưu được sử dụng để dự đoán trên tập dữ liệu và đánh giá bằng hàm OS_ML.

```
xgb = XGBRegressor(random_state=42)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.05, 0.1],
    'subsample': [0.8, 1.0]
}
grid_xgb = GridSearchCV(
    xgb,
    param_grid=param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)
grid_xgb.fit(X_train_final, y_train)
print('Best hyperparameters:', grid_xgb.best_params_)
print('Best CV RMSE:', -grid_xgb.best_score_)

best_xgb = grid_xgb.best_estimator_
results_xgb = OS_ML(
    y_train, best_xgb.predict(X_train_final),
    y_test, best_xgb.predict(X_test_final),
    name='XGBoost'
)
print(results_xgb)
```

Đối với Random Forest, nghiên cứu áp dụng chiến lược tối ưu hóa hai bước nhằm tìm kiếm hiệu quả hơn trong không gian tham số lớn. Trước tiên, RandomizedSearchCV được sử dụng để tìm kiếm ngẫu nhiên các tham số quan trọng như số cây (n_estimators), độ sâu tối đa (max_depth), số lượng biến xem xét

tại mỗi nút (max_features), cũng như các tham số kiểm soát phân tách như min_samples_split và min_samples_leaf.

```
rf = RandomForestRegressor(random_state=42, n_jobs=-1)
param_dist = {
    'n_estimators': randint(100, 500),
    'max_depth': randint(5, 30),
    'max_features': ['sqrt', 'log2'],
    'min_samples_leaf': randint(1, 5),
    'min_samples_split': randint(2, 10)
}
rs_rf = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=30,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    random_state=42
)
rs_rf.fit(X_train_final, y_train)
print('Best params (Random Search):', rs_rf.best_params_)
print('Best CV RMSE:', -rs_rf.best_score_)
```

Sau đó, kết quả từ bước này được sử dụng làm cơ sở để xây dựng một GridSearchCV tinh chỉnh lại các tham số xung quanh giá trị tối ưu đã tìm được. Phương pháp này giúp cân bằng giữa hiệu quả tìm kiếm và chi phí tính toán. Mô hình Random Forest cuối cùng được đánh giá tương tự như các mô hình khác.

```
best_rs = rs_rf.best_params_
param_grid = {
    'n_estimators': [best_rs['n_estimators']],
    'max_depth': [best_rs['max_depth'] - 2, best_rs['max_depth'],
best_rs['max_depth'] + 2],
    'min_samples_split': [best_rs['min_samples_split'] - 2,
best_rs['min_samples_split'], best_rs['min_samples_split'] + 2],
    'min_samples_leaf': [best_rs['min_samples_leaf'] - 1,
best_rs['min_samples_leaf'], best_rs['min_samples_leaf'] + 1],
    'max_features': [best_rs['max_features']]
}
```

```

grid_rf = GridSearchCV(
    rf,
    param_grid=param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)
grid_rf.fit(X_train_final, y_train)
print('Best params (Grid Search):', grid_rf.best_params_)
print('Best CV RMSE:', -grid_rf.best_score_)

best_rf = grid_rf.best_estimator_
results_rf = OS_ML(
    y_train, best_rf.predict(X_train_final),
    y_test, best_rf.predict(X_test_final),
    name='Random Forest (Double Search)'
)
print(results_rf)

```

Sau khi huấn luyện tất cả các mô hình, kết quả đánh giá được tổng hợp và so sánh bằng cách sử dụng hàm `pd.concat()`, cho phép quan sát trực tiếp hiệu suất của từng mô hình trên cùng một bảng. Điều này giúp xác định mô hình nào hoạt động tốt nhất trên bộ dữ liệu.

```

results_all = pd.concat({
    'Linear Regression': results_lr,
    'Ridge': results_ridge,
    'Lasso': results_lasso,
    'XGBoost': results_xgb,
    'Random Forest': results_rf
})
print(results_all)

```

Cuối cùng, để kiểm tra tính ổn định của mô hình tốt nhất (XGBoost), phương pháp cross-validation được áp dụng thêm một lần nữa với 5 folds. Giá trị RMSE trung bình từ cross-validation được so sánh với RMSE trên tập kiểm tra nhằm đánh giá khả năng tổng quát hóa. Nếu hai giá trị này không chênh lệch đáng kể, có thể kết luận rằng mô hình không bị quá khớp và có độ ổn định cao khi áp dụng trên dữ liệu mới.

```

cv_scores = cross_val_score(
    best_xgb,
    X_train_final,
    y_train,
    scoring='neg_root_mean_squared_error',
    cv=5
)
cv_rmse = -cv_scores
print("CV RMSE từng fold:", cv_rmse)
print("Cross-Validation RMSE:", cv_rmse.mean())

```

4.3. Tối ưu siêu tham số

Tối ưu siêu tham số là quá trình tìm kiếm bộ tham số tối ưu cho một mô hình để đạt được hiệu suất dự báo tốt nhất trên dữ liệu chưa biết. Quá trình này thường kết hợp các kỹ thuật tìm kiếm với kiểm chéo để đảm bảo tính tổng quát của mô hình.

4.3.1. Cross-validation

Kiểm chéo là một quy trình trong đó tập dữ liệu huấn luyện được chia thành k tập hợp nhỏ hơn (gọi là các fold). Với mỗi fold, một mô hình sẽ được huấn luyện bằng cách sử dụng $k-1$ folds làm dữ liệu huấn luyện và kết quả sau đó được xác thực trên phần dữ liệu còn lại (fold được giữ lại). Điểm hiệu suất cuối cùng là giá trị trung bình của các giá trị tính được trong vòng lặp.

Việc học các tham số và kiểm tra trên cùng một dữ liệu là một sai lầm phương pháp luận, mô hình có thể đạt điểm hoàn hảo nhưng không dự báo được gì hữu ích trên dữ liệu mới. Nó giúp ước lượng phạm vi điểm số mong đợi của mô hình trên dữ liệu chưa từng thấy. So với việc chia cố định một tập xác thực, kiểm chéo không lãng phí quá nhiều dữ liệu, điều này đặc biệt quan trọng khi số lượng mẫu nhỏ.

Khi điều chỉnh siêu tham số, nếu chỉ dùng một tập kiểm tra duy nhất, kiến thức về tập đó có thể "rò rỉ" vào mô hình, khiến các số liệu đánh giá không còn phản ánh đúng hiệu suất tổng quát.

Hàm sử dụng:

`cross_val_score`: Cách đơn giản nhất để tính toán các chỉ số kiểm chéo trên một estimator.

`cross_validate`: Cho phép chỉ định nhiều chỉ số đánh giá và trả về một từ điển chứa thời gian khớp, thời gian ghi điểm, v.v.

`cross_val_predict`: Trả về dự đoán cho từng phần tử khi phần tử đó nằm trong tập kiểm tra của quá trình kiểm chéo.

4.3.2. *RandomizedSearchCV*

Đây là phương pháp tìm kiếm ngẫu nhiên các siêu tham số, trong đó thay vì thử nghiệm mọi giá trị có thể, hệ thống sẽ lấy mẫu một số lượng cài đặt tham số cố định từ các phân phối được chỉ định. Số lượng cài đặt được thử nghiệm được xác định bởi tham số `n_iter`.

`n_iter` cho phép cân bằng giữa thời gian chạy và chất lượng của giải pháp. Khi không gian tham số quá lớn, tìm kiếm ngẫu nhiên thường hiệu quả hơn về mặt tính toán so với tìm kiếm lưới vét cạn. Khuyến dùng các phân phối liên tục cho các tham số liên tục để việc lấy mẫu đạt hiệu quả cao hơn.

Hàm sử dụng:

`RandomizedSearchCV (estimator, param_distributions, n_iter = 10, random_state = None,...)`.

Các tham số chính:

`param_distributions`: Từ điển chứa tên tham số và các phân phối hoặc danh sách các giá trị để thử.

`n_iter`: Số lượng cài đặt tham số được lấy mẫu.

Các cơ chế tương tự `GridSearchCV`: Nó cũng hỗ trợ các phương thức như `fit`, `predict`, `score` và các thuộc tính như `best_params_`, `best_score_` sau khi hoàn thành.

4.3.3. *GridSearchCV*

Đây là một phương pháp tìm kiếm toàn diện trên các giá trị tham số được chỉ định cho một mô hình. Nó tối ưu hóa các tham số thông qua quá trình tìm kiếm trên lưới kết hợp với kiểm chéo.

Dùng `GridSearchCV` giúp tìm ra tổ hợp siêu tham số tốt nhất một cách tự động thay vì thử nghiệm thủ công. Đảm bảo rằng mọi tổ hợp tham số trong lưới được xác định đều sẽ được thử nghiệm và đánh giá.

Hàm sử dụng:

GridSearchCV(estimator, param_grid, scoring=None, cv=None, refit=True, ...).

Các tham số chính:

param_grid: Từ điển chứa tên tham số và danh sách các cài đặt cần thử.

refit: Nếu là True, nó sẽ huấn luyện lại mô hình trên toàn bộ tập dữ liệu bằng bộ tham số tốt nhất tìm được.

Các thuộc tính sau khi khớp: best_params_ (bộ tham số tốt nhất), best_score_ (điểm số cao nhất), best_estimator_ (mô hình tối ưu nhất).

PHẦN V. ĐÁNH GIÁ MÔ HÌNH

5.1. Thước đo đánh giá

5.1.1. Root Mean Squared Error (RMSE) - Căn bậc hai sai số bình phương trung bình

RMSE là thước đo sự khác biệt giữa các giá trị dự đoán của một mô hình và các giá trị thực tế quan sát được từ môi trường đang được lập mô hình. RMSE đại diện cho độ lệch chuẩn của các sai số dự báo. Nó cho biết mức độ tập trung của dữ liệu xung quanh đường hồi quy.

$$RMSE(X, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2}$$

Trong đó:

m : Là số lượng các thực thể trong tập dữ liệu.

$x^{(i)}$: Là vector chứa tất cả các giá trị thuộc tính của thực thể thứ i .

$y^{(i)}$: Là giá trị mục tiêu (nhãn) thực tế của thực thể thứ i .

h : Là hàm dự đoán của hệ thống (còn gọi là giả thuyết). Khi hệ thống nhận được vector thuộc tính $x^{(i)}$, nó sẽ trả về giá trị dự đoán $y^{(i)} = h(x^{(i)})$.

$RMSE(X, h)$: Là hàm chi phí (cost function) được đo lường trên các thực thể sử dụng giả thuyết h .

RMSE có cùng đơn vị với giá trị mục tiêu cần dự báo. Ví dụ, nếu bạn đang dự báo giá nhà theo đơn vị USD, thì RMSE cũng sẽ có đơn vị là USD, giúp việc giải thích kết quả trở nên trực quan hơn. Do các sai số được bình phương trước khi lấy trung bình, RMSE sẽ phạt nặng hơn các sai số lớn. Điều này có nghĩa là một sai số lớn duy nhất có thể làm tăng đáng kể giá trị RMSE, khiến nó trở nên nhạy cảm với các giá trị ngoại lai. Trong huấn luyện mô hình như hồi quy tuyến tính, mục tiêu thường là tìm các tham số mô hình giúp tối thiểu hóa giá trị RMSE (hoặc MSE - Mean Squared Error). RMSE thường được ưu tiên hơn so với MAE (Mean Absolute Error) khi các sai số lớn đặc biệt không mong muốn. Tuy nhiên, nếu tập dữ liệu có rất nhiều giá trị ngoại lai gây nhiễu, MAE có thể là một thước đo ổn định hơn.

5.1.2. Mean Absolute Error (MAE) - Sai số tuyệt đối trung bình

MAE được tính bằng trung bình trị tuyệt đối của các sai số (phần dư) giữa giá trị dự báo và giá trị thực tế. Vì vậy MAE càng nhỏ mô hình càng dự báo tốt.

$$MAE(X, h) = \frac{1}{m} \sum_{i=1}^m |h(x^{(i)}) - y^{(i)}|$$

Cả RMSE và MAE đều là những cách để đo lường khoảng cách giữa hai vector: vector dự đoán và vector giá trị mục tiêu. Có nhiều phép đo khoảng cách, hay còn gọi là chuẩn khác nhau:

Việc tính toán căn bậc hai của tổng các bình phương (RMSE) tương ứng với chuẩn Euclidean: Đây là khái niệm về khoảng cách mà tất cả chúng ta đều quen thuộc. Nó còn được gọi là chuẩn l_2 , ký hiệu là $\| \cdot \|_2$ (hoặc chỉ là $\| \cdot \|$).

Việc tính toán tổng các giá trị tuyệt đối (MAE) tương ứng với chuẩn l_1 , ký hiệu là $\| \cdot \|_1$. Chuẩn này đôi khi được gọi là chuẩn Manhattan vì nó đo khoảng cách giữa hai điểm trong một thành phố mà bạn chỉ có thể di chuyển dọc theo các khối nhà vuông góc.

Tổng quát hơn, chuẩn l_k của một vector v chứa n phần tử được định nghĩa là $\|v\|_k = (|v_1|^k + |v_2|^k + \dots + |v_n|^k)^{1/k}$. Chuẩn l_0 cho biết số lượng các phần tử khác không trong vector, và chuẩn l_∞ cho biết giá trị tuyệt đối lớn nhất trong vector.

Chỉ số chuẩn càng cao thì nó càng tập trung vào các giá trị lớn và bỏ qua các giá trị nhỏ. Đây là lý do tại sao RMSE nhạy cảm với các giá trị ngoại lai hơn so với MAE. Tuy nhiên, khi các giá trị ngoại lai cực kỳ hiếm xuất hiện (giống như trong biểu đồ hình chuông/phân phối chuẩn), RMSE hoạt động rất tốt và thường được ưu tiên lựa chọn.

5.1.3. R-squared (R^2) - Hệ số xác định

R-squared cho ta biết mức độ các biến đầu vào (biến đầu vào) sẽ giải thích được bao nhiêu phần trăm các biến mục tiêu. R-squared càng lớn thì mô hình càng tốt, khi R-squared bằng 95% điều đó có nghĩa rằng các biến đầu vào đã giải thích được 95% sự biến động của biến mục tiêu.

R-squared được xây dựng dựa trên ba chỉ số:

$$TSS = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$ESS = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

Trong đó TSS là tổng bình phương sai số toàn bộ mô hình (Total Sum Squared), RSS là tổng bình phương sai số ngẫu nhiên (Residual Sum Squared), ESS là tổng bình phương sai số được giải thích bởi mô hình (Explained Sum Squared)

$$R^2 = 1 - \frac{RSS}{TSS}$$

Ý nghĩa giá trị:

R^2 thường nằm trong khoảng từ 0 đến 1. Giá trị càng gần 1, mô hình càng giải thích được nhiều sự biến động của dữ liệu.

$R^2 = 1$ nghĩa là mô hình khớp hoàn hảo với dữ liệu.

R^2 có thể âm trên tập dữ liệu kiểm tra, điều này xảy ra khi mô hình dự báo tệ hơn cả việc lấy giá trị trung bình của dữ liệu để làm dự báo.

Như vậy R^2 càng lớn thì giá trị tổng bình phương sai số càng nhỏ.

5.2. Kết quả dự báo

Tôi đã triển khai năm mô hình phổ biến trong học máy: Linear Regression, Ridge, Lasso, XGBoost và Random Forest, nhằm dự báo doanh số bán hàng tại hệ thống Big Mart. Các chỉ số đánh giá bao gồm MAE, RMSE, R^2 và Bias (Sai số trung bình), giúp đánh giá chính xác khả năng dự báo và độ ổn định của từng mô hình.

Các mô hình học máy dạng ensemble như XGBoost và Random Forest cho thấy khả năng dự báo vượt trội so với các mô hình hồi quy tuyến tính truyền thống. Trong đó, XGBoost được xác định là mô hình tối ưu nhất với sai số thấp nhất và khả năng giải thích dữ liệu cao nhất trên tập kiểm tra. Ngược lại, các mô hình tuyến tính (Linear Regression, Ridge, Lasso) cho kết quả kém hơn đáng kể, cho thấy chúng không đủ mạnh để nắm bắt các mối quan hệ phi tuyến phức tạp trong dữ liệu bán lẻ.

		MAE	RMSE	R2	Bias
Linear Regression	Train	849.001510	1141.034800	0.559767	1.073222e-13
	Test	810.312876	1099.052216	0.568755	-1.200188e+01
Ridge	Train	848.890000	1141.050000	0.560000	0.000000e+00
	Test	810.340000	1099.110000	0.570000	-1.209000e+01
Lasso	Train	848.920000	1141.040000	0.560000	-0.000000e+00
	Test	810.270000	1099.020000	0.570000	-1.203000e+01
XGBoost	Train	750.310000	1061.620000	0.620000	-1.180000e+00
	Test	737.680000	1053.310000	0.600000	-3.219000e+01
Random Forest	Train	677.000000	954.340000	0.690000	6.700000e-01
	Test	756.410000	1070.060000	0.590000	-4.306000e+01

Hình 5.1. Kết quả dự báo tổng hợp của các mô hình

Mô hình cơ bản Linear Regression có kết quả MAE test = 810.31, RMSE test = 1099.05, R^2 test = 0.569, Bias test = -12.0. Mô hình tuyến tính cơ bản này giải thích được khoảng 57% (R^2 test = 0.569) phương sai doanh số, dự báo khá ổn định, chênh lệch giữa train và test nhỏ (Bias test = -12.0), vậy mô hình không bị overfitting. Tuy nhiên, Linear Regression không nắm bắt được các mối quan hệ phi tuyến giữa các đặc trưng sản phẩm, dẫn đến lỗi dự báo trung bình hơi cao. Bias âm chứng tỏ mô hình có xu hướng dự báo thấp hơn thực tế.

Mô hình Ridge Regression cho kết quả MAE test = 810.34, RMSE test = 1099.11, R^2 test = 0.570, Bias test = -12.09. Kết quả gần như Linear Regression vì Ridge chỉ thêm phương pháp chuẩn hoá L2 để giảm overfitting. Mô hình có kết quả ổn định dễ triển khai và giải thích, khả năng dự báo tương đương Linear Regression, cho thấy mô hình vẫn chưa xử lý tốt các quan hệ phi tuyến.

Mô hình Lasso Regression nhận được kết quả MAE test = 810.27, RMSE test = 1099.02, R^2 test = 0.570, Bias test = -12.03. Lasso áp dụng phương pháp chuẩn hoá L1 giúp giảm bớt các biến không quan trọng. Tuy nhiên, do dữ liệu có nhiều biến quan trọng, hiệu quả dự báo không cải thiện đáng kể so với Ridge hay Linear. Mô hình vẫn dự báo hơi thấp hơn thực tế.

Mô hình XGBoost chạy ra kết quả MAE test = 737.68, RMSE test = 1053.31, R^2 test = 0.600, Bias test = -32.19. Mô hình phi tuyến này cải thiện đáng kể độ chính xác dự báo so với các mô hình tuyến tính (MAE giảm tương đương 100). XGBoost học được các quan hệ phi tuyến và tương tác giữa các biến, giúp giải thích tốt hơn sự biến động doanh số. Chênh lệch giữa train và test không quá lớn như vậy

overfitting tương đối thấp. Bias âm nhẹ thể hiện mô hình dự báo hơi thấp, nhưng vẫn nằm trong phạm vi chấp nhận được.

Random Forest có kết quả MAE test = 756.41, RMSE test = 1070.06, R^2 test = 0.590, Bias test = -43.06. Random Forest có khả năng học phi tuyến tốt, nhưng train MAE = 677.0 và R^2 train = 0.69 cho thấy mô hình học quá kỹ trên dữ liệu huấn luyện cho thấy mô hình xuất hiện overfitting nhẹ. Test R^2 giảm còn 0.59, bias âm nhất trong các mô hình dự báo thấp hơn thực tế. Hiệu quả tổng thể mô hình này kém hơn mô hình XGBoost.

Tổng hợp các kết quả trên, XGBoost được xác định là mô hình tối ưu nhất cho bài toán dự báo doanh số tại Big Mart, nhờ khả năng xử lý quan hệ phi tuyến, duy trì độ chính xác cao trên tập kiểm tra và hạn chế overfitting. Các mô hình tuyến tính có thể sử dụng khi cần giải thích dễ hiểu hoặc dữ liệu ít biến, nhưng hiệu quả dự báo kém hơn.

5.3. Kiểm tra độ ổn định của mô hình

Để đánh giá khả năng khái quát và độ ổn định của các mô hình dự báo, chúng tôi tiến hành Cross-Validation 5-fold trên tập huấn luyện và tính RMSE trung bình. Kết quả RMSE của từng fold như sau:

CV RMSE từng fold: [1057.47950144 1083.18362034 1130.41450056 1062.78928255 1127.95016825] Cross-Validation RMSE: 1092.3634146259205

Hình 5.2. Kết quả kiểm tra độ ổn định của mô hình

RMSE trung bình (Cross-Validation RMSE) đạt 1092.36, gần với RMSE tính trên tập kiểm tra (test RMSE) = 1053.31 đối với XGBoost cho thấy mô hình không bị overfitting nghiêm trọng.

CV RMSE: 1092.3634146259205 Test RMSE: 1053.31

Hình 5.3. So sánh RMSE Cross-Validation và RMSE Test của XGBoost

Chênh lệch nhỏ giữa các fold (khoảng từ 1057 – 1128) cho thấy mô hình khá ổn định, không phụ thuộc quá nhiều vào một phân chia dữ liệu cụ thể.

Kết quả kiểm tra độ ổn định khẳng định XGBoost là một mô hình dự báo ổn định và tin cậy, phù hợp để áp dụng trong thực tiễn dự báo doanh số bán hàng tại hệ thống Big Mart.

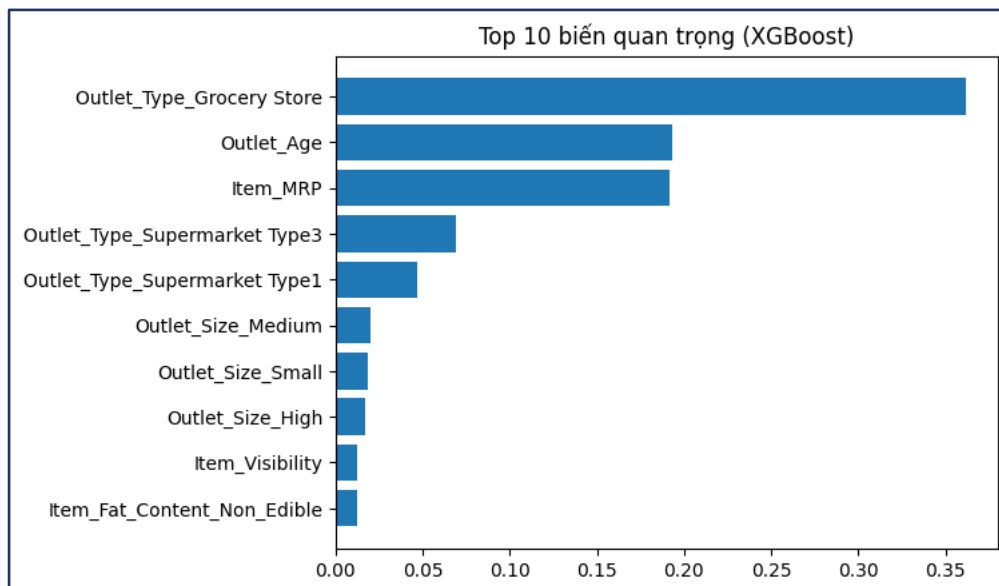
PHẦN VI. PHÂN TÍCH CÁC YẾU TỐ ẢNH HƯỞNG ĐẾN DOANH SỐ

6.1. Phân tích độ quan trọng của biến

Sau khi xây dựng và huấn luyện mô hình XGBoost, bước tiếp theo là phân tích độ quan trọng của các biến để xác định những yếu tố có ảnh hưởng mạnh nhất đến doanh số bán lẻ. Đây là bước then chốt giúp hiểu “tư duy” của mô hình, tức là mô hình dựa vào những biến nào nhiều nhất để đưa ra dự báo, đồng thời hỗ trợ doanh nghiệp xác định các điểm cần tập trung cải thiện.

Trong biểu đồ cột ngang mà tôi sử dụng, trục X (chiều ngang) đại diện cho giá trị độ quan trọng của từng biến do XGBoost tính toán, trong khi trục Y (chiều dọc) hiển thị tên biến. Giá trị importance càng cao nghĩa là biến đó đóng góp càng nhiều vào độ chính xác của mô hình. Lệnh sử dụng cụ thể như sau:

```
importance = best_xgb.feature_importances_  
features = X_train_final.columns  
idx = np.argsort(importance)[-10:]  
plt.figure()  
plt.barh(range(len(idx)), importance[idx])  
plt.yticks(range(len(idx)), [features[i] for i in idx])  
plt.title("Top 10 biến quan trọng (XGBoost)")  
plt.show()
```



Hình 6.1. Top 10 biến quan trọng đến doanh thu

Dựa trên kết quả phân tích, 10 biến hàng đầu có thể chia thành ba nhóm:

- **Nhóm ảnh hưởng mạnh dẫn đầu:**

Outlet_Type_Grocery_Store (~0.36): Đây là biến ảnh hưởng mạnh nhất. Kết quả này cho thấy sự khác biệt rõ rệt về hiệu suất hoặc hành vi mua sắm tại các cửa hàng tạp hóa so với các loại hình khác. Điều này đồng nghĩa rằng loại hình cửa hàng tạp hóa đóng vai trò đặc biệt quan trọng và mạnh mẽ hơn các nhóm cửa hàng khác trong việc tạo ra doanh thu, từ đó nhấn mạnh tầm ảnh hưởng chủ đạo của yếu tố này trong chiến lược kinh doanh và dự báo doanh số bán lẻ.

- **Nhóm ảnh hưởng chính:**

Outlet_Age (~0.19): Thời gian hoạt động của cửa hàng là yếu tố quan trọng, ảnh hưởng trực tiếp đến doanh số. Cụ thể, các cửa hàng lâu năm thường xây dựng được uy tín và thương hiệu ổn định, từ đó thu hút và giữ chân khách hàng trung thành. Điều này dẫn đến doanh số ổn định hơn so với các cửa hàng mới mở, vì khách hàng có xu hướng quay lại các địa điểm họ đã tin tưởng. Ngoài ra, tuổi đời cửa hàng cũng có thể phản ánh kinh nghiệm quản lý và hiệu quả vận hành, từ đó cải thiện hiệu quả bán hàng.

Item_MRP (~0.19): Giá bán lẻ tối đa là một chỉ số kinh tế then chốt, ảnh hưởng trực tiếp đến doanh thu. Cụ thể, sản phẩm có MRP cao thường mang lại biên lợi nhuận lớn hơn, nhưng cũng cần cân nhắc sức mua của khách hàng. Ngược lại, MRP thấp có thể tạo sức hút cho lượng khách hàng lớn hơn, nhưng doanh thu trên mỗi sản phẩm thấp. Vì vậy, giá bán lẻ tối đa là yếu tố mà mô hình XGBoost dựa vào nhiều để dự báo doanh số, phản ánh mối quan hệ giữa chiến lược giá và kết quả kinh doanh thực tế.

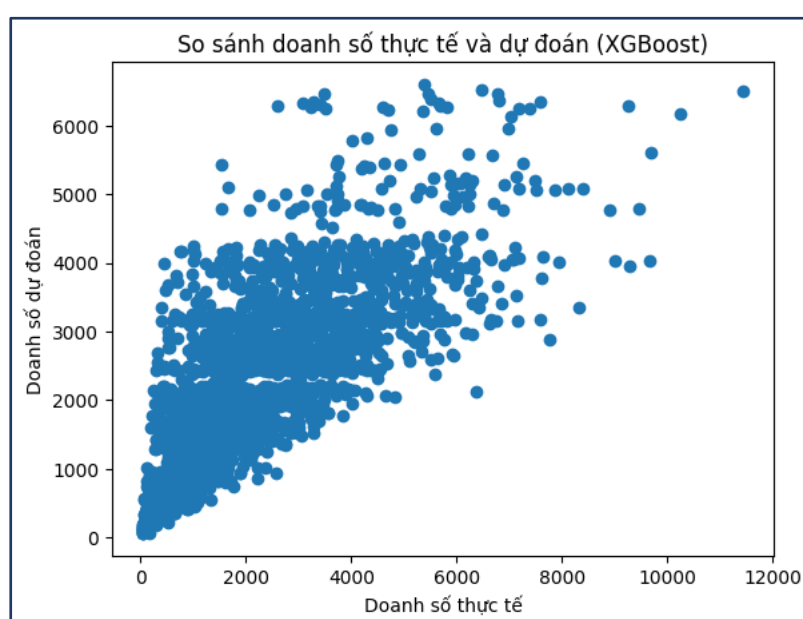
- **Nhóm ảnh hưởng bổ trợ:**

Các biến liên quan đến loại hình siêu thị (Supermarket Type3, Supermarket Type1) và quy mô cửa hàng (Outlet_Size) tuy có tác động thấp hơn so với các biến nhóm dẫn đầu và nhóm chính, nhưng vẫn đóng vai trò nhất định trong việc dự báo doanh số. Cụ thể, loại hình siêu thị và quy mô cửa hàng có thể ảnh hưởng đến sức chứa sản phẩm, khả năng phục vụ khách hàng và trải nghiệm mua sắm, từ đó gián tiếp tác động đến doanh thu. Mặc dù mức ảnh hưởng không quá lớn, nhưng việc cân nhắc các yếu tố này vẫn giúp mô hình dự báo chính xác hơn.

Các đặc tính sản phẩm như Item_Visibility (độ hiển thị sản phẩm) và Item_Fat_Content có tác động rất nhỏ đến dự báo doanh số. Điều này cho thấy trong mô hình XGBoost, các đặc điểm của sản phẩm đóng vai trò hỗ trợ hơn là yếu tố quyết định. Việc này cũng phản ánh thực tế bán lẻ, khi doanh số chủ yếu bị chi phối bởi yếu tố cửa hàng và giá cả hơn là các đặc tính phụ của sản phẩm.

6.2. So sánh giá trị thực tế và dự đoán

Sau khi có mô hình dự báo tốt, tôi tiến hành so sánh **giá trị doanh số thực tế và dự báo** từ mô hình XGBoost. Biểu đồ phân tán thể hiện mối quan hệ giữa doanh số thực tế (X) và dự báo (Y) như sau:



Hình 6.2. So sánh doanh số thực và dự đoán

Tôi nhận thấy mô hình XGBoost thể hiện khả năng dự báo khá tốt, đồng thời cũng bộc lộ một số hạn chế rõ ràng. Biểu đồ cho thấy các điểm dữ liệu tập trung thành một dải đi lên theo đường chéo, phản ánh xu hướng thuận, tức là mô hình đã học được quy luật chung và dự báo có độ tương quan khá tốt với giá trị thực tế. Ở phân khúc doanh số thấp và trung bình (0–4,000), các điểm dữ liệu hội tụ sát nhau, cho thấy mô hình dự báo ổn định và chính xác đối với những sản phẩm hoặc cửa hàng có doanh thu vừa và nhỏ. Tuy nhiên, khi doanh số thực tế vượt quá 6,000, các điểm bắt đầu phân tán rộng hơn và có xu hướng nằm dưới đường chéo lý tưởng, đặc biệt mô hình dường như “chặn” ở mức dự báo khoảng 6,500, trong khi một số giá trị thực tế lên tới hơn 10,000.

Nguyên nhân dẫn đến hiện tượng này có thể được giải thích từ nhiều khía cạnh. Trước hết, sự hiện diện của các giá trị ngoại lai trong biến mục tiêu `Item_Outlet_Sales` là rất rõ, với khoảng cách lớn giữa tứ phân vị 75% (khoảng 3101) và giá trị tối đa (13086), khiến mô hình khó dự báo chính xác các giá trị cực đại. Về bản chất của thuật toán XGBoost và hàm mục tiêu có thành phần hiệu chuẩn nhằm ngăn hiện tượng quá khớp, dẫn đến mô hình ưu tiên các dự báo an toàn gần giá trị trung bình thay vì theo đuổi các giá trị doanh số cực cao, từ đó gây ra sai số âm ở các mức doanh số lớn. Về vấn đề thiếu hụt các biến giải thích cho sự đột biến, chẳng hạn như yếu tố thời gian, chu kỳ lễ tết, hoặc các chương trình khuyến mãi đặc biệt, khiến mô hình không có cơ sở để dự đoán các mức doanh số cực đại mà chỉ dựa vào các thuộc tính hiện có như loại hình cửa hàng (`Outlet_Type`) và giá niêm yết (`Item_MRP`). Cuối cùng, các biến phụ trợ như `Item_Visibility` có nhiều giá trị ngoại lai nhưng đóng vai trò rất nhỏ trong mô hình (~ 0.01), nên các trường hợp sản phẩm được ưu tiên trưng bày để tạo đột phá doanh thu không được dự báo chính xác.

Tóm lại, kết quả phân tích cho thấy mô hình XGBoost hoạt động ổn định và tin cậy đối với các hoạt động bán lẻ thông thường, đặc biệt ở phân khúc doanh số thấp và trung bình. Tuy nhiên, để cải thiện khả năng dự báo cho các trường hợp doanh số cao đột biến, chúng tôi nhận thấy cần bổ sung thêm các thông tin về yếu tố thời gian, khuyến mãi hoặc sự kiện đặc biệt, nhằm tăng khả năng phản ánh các biến động bất thường trong dữ liệu bán lẻ.

PHẦN VII. NGUỒN TÀI LIỆU THAM KHẢO

- [1] Ali, Z. A., Abduljabbar, Z. H., Tahir, H. A., Sallow, A. B., & Almufti, S. M. (2023). *Exploring the Power of eXtreme Gradient Boosting Algorithm in Machine Learning: A Review*. Academic Journal of Nawroz University, 12(2).
<https://doi.org/10.25007/ajnu.v12n2a1612>.
- [2] Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. arXiv preprint arXiv:1603.02754.
<https://arxiv.org/pdf/1603.02754>.
- [3] Random Forests. Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>.
- [4] Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras, and tensorflow* (3rd ed.). O'Reilly Media.
- [5] scikit-learn. (n.d.). *RandomizedSearchCV* [API documentation]. In *scikit-learn: Machine learning in Python*.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html.
- [6] scikit-learn. (n.d.). *Cross-validation: evaluating estimator performance* [Documentation]. In *scikit-learn: Machine learning in Python*. Retrieved April 2, 2026, from https://scikit-learn.org/stable/modules/cross_validation.html.
- [7] scikit-learn. (n.d.). *GridSearchCV* [API documentation]. In *scikit-learn: Machine learning in Python*. Retrieved April 2, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html.

